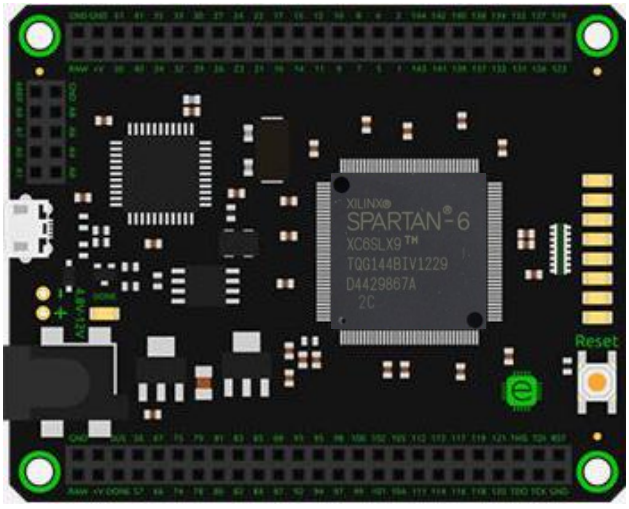
دانشکده مهندسی

گروه مهندسی برق

دستور کار آزمایشگاه سیستم‌های دیجیتال ۲

نویسنده: مهندس سید محمد عجمی بهبهانی

مهر ۱۴۰۴ (ویرایش ۳)



پیش‌گفتار

هدف از درس آزمایشگاه سیستم‌های دیجیتال ۲، آشنایی با نحوه‌ی طراحی و پیاده‌سازی مدارهای دیجیتال بر روی تراشه‌ی FPGA است. بدین منظور، ابتدا ساختار داخلی این تراشه شرح داده می‌شود. سپس به کمک نرم‌افزار Xilinx ISE 14.7، دانشجویان نحوه‌ی طراحی مدارهای دیجیتال را به صورت‌های شماتیک و زبان توصیف سخت‌افزار VHDL می‌آموزند. در انتها، دانشجویان مدارهای توصیف شده را بر روی برد موجود در آزمایشگاه پیاده‌سازی می‌کنند.

نحوه‌ی ارزشیابی دانشجویان به سه بخش تقسیم می‌شود. بخش اول مربوط به انضباط، حضور به موقع در کلاس درس و رعایت قوانین آزمایشگاه است. بخش دوم مربوط به انجام آزمایش‌های هفتگی، ارائه‌ی گزارش‌کار و تمرین‌هاست که دانشجویان باید در زمان خواسته شده، آن‌ها را تحویل دهند. بخش سوم مربوط به امتحان تئوری و پروژه‌های درسی است. بنابراین نمرات درسی دانشجویان به صورت زیر محاسبه می‌شود:

۱) گزارش‌کار گروهی و تمرین	**۳ نمره**
۲) حضور و غیاب و نظم در آزمایشگاه	**۳ نمره**
۳) کار گروهی در آزمایشگاه	**۵ نمره**
٤) آزمون عملی (یا پروژه)	**۵ نمره**
۵) آزمون تئوری	**٤ نمره**

نمونه‌ی گزارش‌کار و تمرین

بسمه تعالی	
عنوان: شماره‌ی آزمایش/تمرین	
هدف آزمایش/تمرین:	
نام و نام‌خانوادگی:	شماره دانشجویی:
نام اعضای گروه:	

قوانین ارائه‌ی گزارش‌کار و تمرین:

۱) **هر دو در برگه سایز A4 تحویل داده می‌شوند.**

۲) **اگر به صورت تایپ شده هستند، باید حتما قواعد زیر را در نوشتار آن‌ها رعایت کنید:**

a. **متن اصلی گزارش با فونت B Lotus و سایز ۱۲ نوشته شود**

b. **عناوین با همان فونت قبل و با سایز ١٤ به صورت Bold قرار بگیرد**

c. **می‌توانید به جای پرینت گزارش‌کار، آن را به آدرس زیر ایمیل کنید:**

mohammadbeh1994@gmail.com

دقت کنید که در ایمیل ارسالی، حتما مشخصات خود و شماره‌ی تمرین/گزارش را در عنوان ذکر کنید.

۳) **تمام کدهایی که در هر متن شما قرار می‌گیرند، باید خط به خط شرح داده می‌شوند و در آخر، یک نتیجه‌ی کلی از توضیحات گرفته می‌شود. در صورت عدم رعایت این نکته، نمره کسر می‌شود.**

٤) **ملاک نمره‌دهی به گزارش/تمرین، فقط ارائه‌ی آن نیست. علاوه‌بر تحویل به موقع، رعایت علائم نگارشی، خوانا بودن دست‌خط و رعایت نظم ظاهری گزارش‌های شما اهمیت زیادی دارد.**

۵) **در انتهای تمرین/گزارش خود، حتما و حتما مراجع استفاده شده را ذکر کنید.**

بخش اول - مروری بر سیستم‌های دیجیتال

هدف این بخش مروری کلی بر سیستم‌های دیجیتال، ابزارهای پیاده‌سازی آن‌ها و بررسی روش طراحی و پیاده‌سازی این سیستم‌ها بر روی بردهای FPGA[1] موجود در آزمایشگاه است. بدین منظور در فصل اول، مفاهیم پایه در سیستم‌های دیجیتال را مطالعه خواهیم کرد. هدف این فصل مروری بر مباحث مربوط به سیستم اعداد مبنای دو، جبر بول، آشنایی با انواع مدار دیجیتال و روش طراحی مدارهای ترکیبی و ترتیبی می‌باشد. در فصل دوم، با ادوات پیاده‌سازی سیستم‌های دیجیتال آشنا می‌شویم.

[1] Field Programmable Gated Array

فصل ۱ - مروری بر سیستم‌های دیجیتال

۱-۱. مقدمه

هدف این فصل مطالعه‌ی مفاهیم پایه‌ای سیستم‌های دیجیتال است. بدین منظور، ابتدا تعاریف اولیه‌ی این سیستم‌ها را مرور خواهیم کرد. سپس سیستم اعداد باینری، جبر بول، انواع مدارهای دیجیتال و منطق چند ارزشی دوره می‌شود . در بخش بعد، روش طراحی مدارهای ترکیبی و ترتیبی را ذکر می‌کنیم و در انتهای این فصل، با شاخص‌های طراحی و مقایسه‌ی سیستم‌های دیجیتال آشنا می‌شویم.

۱-۲.تعاریف اولیه‌ی سیستم‌های دیجیتال

سیستم‌های دیجیتال دسته‌ای از مدارهای الکتریکی هستند که برخلاف سیستم‌های آنالوگ، هدفشان پردازش سیگنال‌های گسسته است. این مدارها یک یا چند ورودی باینری را دریافت کرده و خروجی یا خروجی‌های باینری تولید می‌کنند. به عبارتی، ورودی و خروجی این مدارها رشته‌ای از اعداد ۰ و ۱ هستند. روابط بین ورودی‌ها/خروجی‌های این سیستم‌ها به کمک جدولی به نام درستی یا نوع خاصی از جبر، به نام جبر بول توصیف می‌شود. اجزای اصلی این سیستم‌ها را گیت‌های پایه‌ی منطقی تشکیل می‌دهند. در یک حالت کلی، این سیستم‌ها به سیستم‌های بدون فیدبک و دارای فیدبک تقسیم می‌شوند. در ادامه‌ی این بخش، این مفاهیم را بیشتر بررسی خواهیم کرد.

۱-۲-۱- سیستم اعداد دودویی (باینری)

در حالت کلی یک عدد در مبنای r به صورت زیر نمایش داده می‌شود:

$$(a_n a_{n-1} \dots a_0)_r = a_n r^{n-1} + a_{n-1} r^{n-2} + \cdots + a_0 \qquad \text{رابطه (۱-۱)}$$

که ضرایب $0 < a_j < r - 1$ می‌باشند. به عنوان نمونه، عدد ٦٧٥ در مبنای ۱۰[۲] با ضرایب توانیش به صورت ۵ + ۱۰×۷ + ۱۰۰×٦ نشان داده می‌شود. در زندگی روزمره، انسان‌ها به طور معمول از سیستم مبنای ۱۰ برای شمارش استفاده می‌کنند. اما عملکرد سیستم‌های دیجیتال براساس اعداد مبنای ۲، یا همان اعداد دودویی است. گاهی به منظور ساده‌سازی نمایش اعداد مبنای ۲، معمول است که از نمایش این اعداد در مبنای ۸ یا ١٦ استفاده شود. به مشابه اعداد مبنای ۱۰، برای اعداد مبنای ۲ نیز عملگرهای ریاضی تعریف می‌شود. در این حالت با دو دسته از عملگرهای منطقی و محاسباتی مواجه هستیم. در بخش بعد، به بررسی جبر بول، که هدف آن تعریف عملگرهای منطقی است برای اعداد دودویی است، می‌پردازیم.

[2] Decimal

۱-۲-۲- جبر بول

جبر بول نوعی از جبر است که بر پایه‌ی منطق دو ارزشی، به معنای درست بودن گزاره یا یک منطقی و غلط بودن گزاره یا صفر منطقی، استوار شده است. متغیرهای دودویی در این نوع از جبر با حروف الفبای انگلیسی، A,B,Z و غیره مشخص می‌شوند و برخلاف جبر عادی، تنها دو مقدار ۰ یا ۱ را اختیار می‌کنند. روابطی که بین متغیرها برقرار است به کمک جدول درستی یا معادلات بولین نشان داده می‌شود. در جبر بول، سه عملگر اصلی به نام‌های NOT، AND و OR معرفی می‌شوند که مشخصات آن‌ها در جدول ۱-۱ نشان داده شده است.

جدول ۱-۱ – عملگرهای پایه‌ی منطقی NOT، AND و OR

عملگر	نماد	معادله‌ی بولی به همراه جدول درستی
NOT	$\bar{}$(´)	$y = \bar{a}$ a y 0 1 1 0
AND	·	$y = a \cdot b$ a b y 0 0 0 0 1 0 1 0 0 1 1 1
OR	+	$y = a + b$ a b y 0 0 0 0 1 1 1 0 1 1 1 1

۱-۲-۲- مدارهای دیجیتال

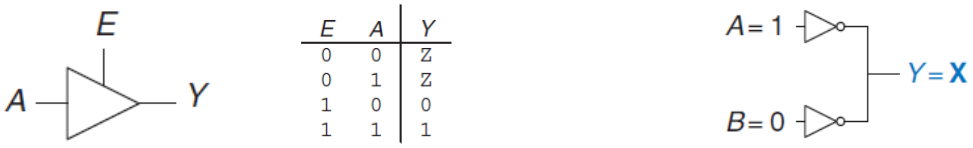
مدارهای دیجیتال به دو دسته‌ی ترکیبی و ترتیبی تقسیم می‌شوند. مدارهایی که خروجی آن‌ها در هر لحظه، وابسته به ورودی‌هایشان فقط در همان لحظه است، مدارهای ترکیبی نام دارند. به عبارتی، خروجی یا خروجی‌های این مدارها، تابع بولین از ورودی‌های آن است. به عنوان نمونه، گیت‌های منطقی، که المان‌های پایه‌ی مدارهای دیجیتال هستند، از نوع مدارهای ترکیبی به حساب می‌آیند. بر خلاف مدارهای ترکیبی، خروجی‌های مدارهای ترتیبی در هر لحظه، وابسته به ورودی‌هایشان در همان لحظه و لحظات قبل است. به عبارتی مدارهای ترتیبی دارای حافظه یا فیدبک هستند. به عنوان مثال، شمارنده، دنبال‌کننده‌ی رشته و تقسیم‌کننده‌ی فرکانسی از انواع مدارهای ترتیبی هستند.

۱-۲-۳- منطق چند ارزشی در مدارهای دیجیتال

با وجود اینکه در جبر بولین تنها مقادیر ۰ و ۱ منطقی تعریف می‌شوند، اما در مدارهای الکتریکی دو مقدار دیگر نیز وجود دارند (تصویر ۱-۱):

X – مقدار نامعلوم یا غیرمجاز– حالتی که خروجی دو یا چند مدار دیجیتال به یک‌دیگر متصل شود. در این حالت مقدار خروجی نامعین است. **این حالت ممکن است موجب آسیب به مدار شود. (نکته: این حالت با حالت بی‌اهمیت یا X در جدول کارنو متفاوت است).**

Z – مدار باز (High Impedance)– حالتی که اتصال یک گره خروجی از مدار جدا شده باشد.



الف–اتصال خروجی دو گیت منطقی–حالت غیرمجاز ب–حالت مدار باز یا **High Impedance**

تصویر (۱-۱) – حالت‌های غیرمجاز و مدار باز[۲]

۱-۳. طراحی مدارهای دیجیتال

در این بخش به طراحی مدارهای دیجیتال به دو روش سطح گیت[۳] و سطح انتقال ثبات[۴] می‌پردازیم.

۱-۳-۱- طراحی مدارهای ترکیبی در سطح گیت

هنگام طراحی مدارهای ترکیبی، باید نکات زیر را به دقت مورد بررسی قرار داد:

۱) مدارهای ترکیبی بدون حافظه هستند– در این مدارها فلیپ‌فلاپ یا لچ وجود ندارد.

۲) مسیر چرخشی در این نوع مدار وجود ندارد – به عبارتی هر گره، ورودی یا خروجی یک گیت است و **پین ورودی–خروجی وجود ندارد.**

به منظور طراحی مدارهای ترکیبی، مراحل زیر را انجام می‌دهیم:

۱) فهم مساله و مشخص کردن ورودی‌ها و خروجی‌های سیستم

۲) ترسیم جدول درستی

۳) استفاده از مینترم‌ها[۵] یا ماکسترم‌های[۶] جدول درستی به منظور نوشتن معادله بولین

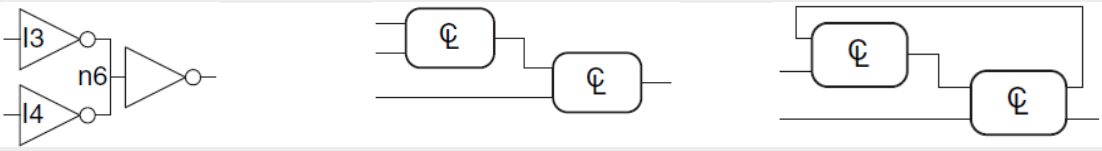
---

[3] Gate Level
[4] Register Transfer Level
[5] Minterm
[6] Maxterm

٤) استفاده از جبر بول یا جدول کارنو برای ساده‌سازی معادله‌ی بولین حاصل شده

مثال ۱

۱. کدام یک از مدارهای زیر ترکیبی هستند[۲] (C_L را یک واحد ترکیبی در نظر بگیرید)



ج ب الف

الف- به علت وجود مسیر چرخشی، مدار ترکیبی نیست

ب- مدار ترکیبی است – مسیر چرخشی ندارد و خروجی‌های هیچ دو گیتی به هم متصل نشده است

ج- خروجی دو گیت به یکدیگر متصل شده است، پس مدار ترکیبی نیست

۲. مالتی‌پلکسر را تعریف کرده و به کمک جدول درستی آن، یک مالتی‌پلکسر ۱×٤ طراحی کنید.

۱-۳-۲- طراحی مدارهای ترتیبی در سطح گیت

در مدارهای ترتیبی دو تعریف مهم وجود دارند که عبارت‌اند از:

۱) حالت (State): مشخص کننده‌ی رفتار سیستم در آینده

۲) حافظه: عناصری که حالت سیستم را ذخیره می‌کنند. به عنوان نمونه، فلیپ-فلاپ و لچ

با توجه به تعاریف بالا، مدارهای ترتیبی به دو دسته‌ی سنکرون[۷] و آسنکرون[۸] تقسیم می‌شوند. در مدارهای سنکرون، از فلیپ فلاپ به عنوان حافظه برای ذخیره‌ی حالت سیستم استفاده می‌شود. در مدارهای آسنکرون از یک المان تاخیر دار، مثلا گیت‌های منطقی، به عنوان ذخیره کننده‌ی حالت سیستم استفاده می‌شود. امروزه بیشتر مدارهای ترتیبی به صورت سنکرون طراحی می‌شوند. لذا هدف ما نیز طراحی مدارهای ترتیبی سنکرون خواهد بود.

به صورت معمول، عنصر ذخیره کننده‌ی حالت در یک مدار ترتیبی سنکرون را فلیپ-فلاپ یا ثبات‌ها تشکیل می‌دهند. در حالت کلی، یک مدار ترتیبی سنکرون دارای ویژگی‌های زیر است:

۱) در حالت کلی، شامل مدارهای ترکیبی و ثبات‌ها هستند.

۲) حداقل دارای یک ثبات است.

[7] Synchronous
[8] Asynchronous

۳) تمام ثبات‌ها با یک سیگنال پالس[9] ساعت به روز می‌شوند.

٤) در یک مسیر بسته، حداقل یک ثبات وجود دارد.

یکی از روش‌های مرسوم برای توصیف مدارهای ترتیبی سنکرون، ماشین حالت است. هر ماشین حالت شامل سه المان پایه است:

۱) مدار ترکیبی برای تعیین حالت بعد

۲) مدار ترکیبی برای تعیین خروجی

۳) ثبات به منظور ذخیره‌ی حالت

مراحل طراحی یک مدار ترتیبی سنکرون به صورت زیر می‌باشد:

۱) توصیف مدار ترتیبی

۲) رسم نمودار حالت

۳) به دست آورن جدول حالت

٤) کاهش حالت

۵) تخصیص حالت

٦) انتخاب نوع فلیپ‌فلاپ و تعیین حالت

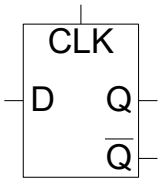
۷) رسم جدول‌های توابع ورودی فلیپ‌فلاپ‌ها و خروجی‌های مدار

۸) ساده‌سازی توابع ورودی فلیپ‌فلاپ‌ها و خروجی‌های مدار

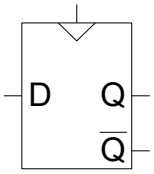
۹) رسم مدار

مثال ۲

الف- با توجه به تصویر زیر[۲]، تفاوت لچ و فلیپ-فلاپ را توضیح دهید.

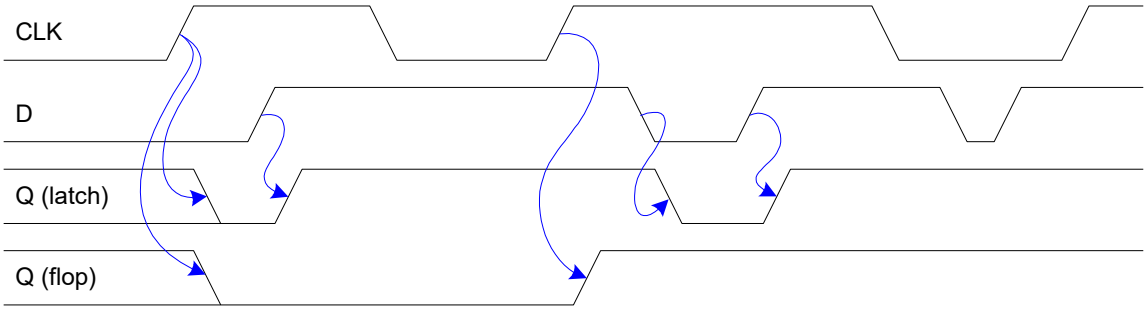


Latch



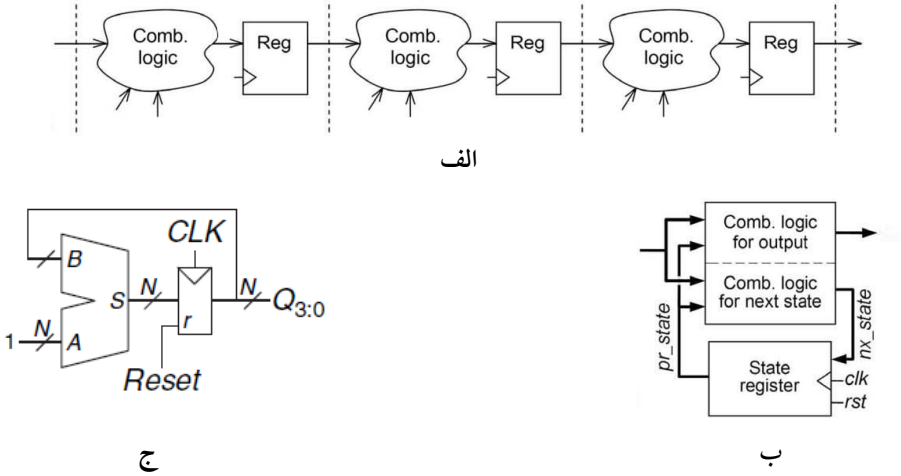
Flip-Flop

[9] Pulse Signal



۱-۳-۳. طراحی مدارهای دیجیتال در سطح توصیف انتقال ثبات (RTL)

در دو بخش قبل در ارتباط با طراحی مدارهای ترکیبی و ترتیبی صحبت شد. در این روش‌ها، اجزای اصلی طراحی را گیت‌های منطقی تشکیل می‌دهند. لذا این روش‌ها را طراحی در سطح گیت می‌نامند. در روش دیگر از طراحی مدارهای دیجیتال، که طراحی در سطح توصیف انتقال ثبات نامیده می‌شود، یک مدار به صورت شبکه‌ای از المان‌های ذخیره کننده (ثبات یا فلیپ فلاپ) است که المان‌های ترکیبی (مانند جمع‌کننده، مالتی پلکسر و مقایسه‌کننده) آن‌ها را به یکدیگر مرتبط می‌کند (تصویر ۱-۳-الف و ۱-۳-ب)[۱]. به عنوان نمونه، همانگونه که در تصویر ۱-۳-ج مشاهده می‌شود، یک شمارنده‌ی ٤ بیت از دو المان جمع کننده و ثبات ٤ بیت شکیل شده است. از مزایای این روش، پیچیدگی کمتر در طراحی مدارهاست.



تصویر ۱-۲ – توصیف مدارهای دیجیتال در سطح RTL [۱]

۱-٤. شاخص طراحی مدارهای دیجیتال

مهم‌ترین مشخصه‌هایی که در تحلیل یک مدار ترتیبی سنکرون بررسی می‌شوند، عبارت‌اند از[۱٤] :

- ابعاد مدار(A[10]): معیاری است به منظور نشان دادن منابع سخت‌افزاری استفاده شده. این شاخص می‌تواند بسته به نوع سخت‌افزار، با معیارهای مختلفی بیان شود. به عنوان نمونه تعداد گیت معادل که برابر با تعداد المان‌های منطقی کامل مورد نیاز مانند NOR و یا NAND است.
- دوره زمانی محاسباتی(T): نشان دهنده‌ی حداقل فاصله‌ی زمانی لازم میان دو داده‌ی ورودی متوالی است.
- نرخ پردازش داده[11](Θ): بیانگر تعداد عمل‌های پردازشی در واحد زمان است و واحد آن به نوع داده‌ای که پردازش می‌شود، بستگی دارد. این مشخصه با مشخصه‌ی T رابطه‌ی عکس دارد.
- شاخص $A \times T$: این شاخص، نشان می‌دهد که چه تعادلی میان میزان منابع سخت‌افزاری مورد استفاده و نرخ پردازش حاصل شده وجود دارد. هرچه اندازه‌ی این شاخص کمتر باشد، معماری سخت‌افزاری بهینه‌تر است. به عبارتی، در مساحت کمتر، نرخ پردازش بیشتری خواهیم داشت.

مثال ۳

شاخص‌های مداری شمارنده‌ی ٤ بیت موجود در تصویر ۱-۳-ج را بنویسید.

$$\boldsymbol{A = A_{+} + 4A_{reg}}$$
$$\boldsymbol{T = t_{+} + t_{reg}}$$

نرخ پردازش: متناسب است با فرکانس کاری مدار که مقدار بیشینه‌ی آن $\frac{1}{T}$ است

تمرین

۱. اعداد زیر را در مبنای ۲، ۸ و ١٦ نشان دهید.

66 21 245 333

۲. اعداد زیر را به مبنای خواسته شده تبدیل کنید.

$(\mathbf{AB87})_{\mathbf{16}} = (\ )_{\mathbf{10}}$ $(\mathrm{AF01})_{16} = (\ )_{8}$ $(1001110)_{2} = (\ )_{16}$ $71 = (\ )_{2}$

۳. در ارتباط با اعداد BCD تحقیق کنید.

٤. سیستم نمایش اعداد علامت‌دار در سیستم باینری به چه صورت است؟ روش متمم دو را توضیح دهید.

۵. انجام محاسبات ریاضی جمع، تفریق و ضرب را بر روی اعداد باینری با ذکر یک مثال توضیح دهید.

٦. در مورد ساده‌سازی معادلات بولی و قواعد آن به کمک جبر بول تحقیق کنید.

۷. تمام گیت‌های منطقی را ذکر کرده و جدول درستی آن را ترسیم کنید.

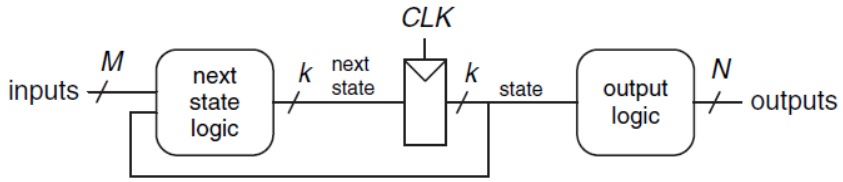
[10] Area
[11] Data throughput

۸. دو مدار ترکیبی زیر را طراحی کنید:

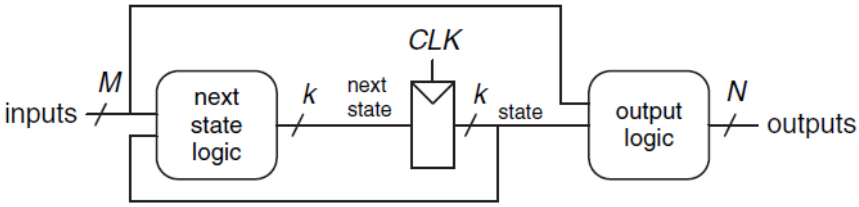
۱. دیکدر BCD به سون سگمنت آند مشترک

۲. مقایسه کننده‌ی ٤ بیت

۹. ماشین حالت میلی و مور را تعریف کرده و آن‌ها را با یکدیگر مقایسه کنید.



الف–ماشین حالت مور [۲]



ب- ماشین حالت میلی [۲]

۱۰. به کمک جدول انتقال حالت، یک شمارنده‌ی ٤ بیت طراحی کنید.

۱۱. در مورد مفاهیم زیر تحقیق کنید:

- تاخیر انتشار[۱۲]
- مسیر ترکیبی[۱۳]
- تاخیر مسیر بحرانی[۱٤]

[12] Propagation delay
[13] Combinational path
[14] Critical path delay

فصل ۲ – ادوات پیاده‌سازی مدارهای دیجیتال

۲-۱. مقدمه

هدف این فصل آشنایی با سخت‌افزارهای متداول به منظور پیاده‌سازی مدارهای دیجیتال است. در حالت کلی، این ابزارها به سه دسته‌ی خاص تقسیم می‌شوند. دسته‌ی اول، تراشه‌هایی هستند که شامل گیت‌های منطقی یا المان‌های بزرگتری مانند حافظه یا واحد حساب و منطق (ALU[15]) می‌شوند. دسته‌ای دیگر، تراشه‌های برنامه‌پذیر هستند که به کمک آن‌ها، یک سیستم دیجیتال با عملکرد دلخواه ایجاد می‌شود. و دسته‌ی آخر، مدارهای مجتمع ASIC[16] هستند که به منظور انجام یک انجام پردازش خاص، مثلا پردازش سیگنال، ساخته شده‌اند. در بخش‌های بعد، هر دسته از این ابزارها به صورت مختصر شرح داده می‌شوند.

۲-۲. تراشه‌های منطقی

این تراشه‌ها شامل گیت‌های پایه‌ی منطقی یا مدارهای منطقی متداول هستند که به صورت معمول از تعداد کمی ترانزیستور ساخته شده‌اند و به آن‌ها مدارهای مجتمع SSI[17] نیز گفته می‌شود. به عنوان نمونه، مدارهای مجتمع سری 74-XX نوعی از این تراشه‌ها محسوب می‌شوند که به عنوان نمونه، شامل گیت‌های منطقی AND(تراشه‌ی ۷٤۰۸)، NOT(تراشه‌ی ۷٤۰٤) و فلیپ فلاپ D(تراشه‌ی ۷٤۷٤) هستند. این تراشه‌ها معمولا در پروژه‌های کلاسی یا سیستم‌های دیجیتالی که پیچیدگی کمی دارند، مورد استفاده قرار می‌گیرند.

۲-۳. تراشه‌های منطقی برنامه‌پذیر

تراشه‌های برنامه‌پذیر شامل آرایه‌هایی از المان‌های برنامه‌پذیر هستند که به منظور انجام یک کاربرد خاص پیکربندی می‌شوند. البته برخلاف میکروپروسسورها، که برنامه‌پذیرند اما سخت‌افزار آن‌ها ثابت می‌ماند، در تراشه‌های برنامه‌پذیر، این سخت‌افزار است که قابلیت پیکربندی دارد. خاصیت برنامه‌پذیری این تراشه‌ها معمولا به علت داشتن فیوزهای برنامه‌پذیر یا حافظه است. به عنوان نمونه، مدارهای مجتمع SPLD[18]، CPLD[19] و FPGA از انواع تراشه‌های برنامه‌پذیر هستند که در ادامه‌ی این بخش، ساختار آن‌ها به صورت خلاصه توضیح داده می‌شود.

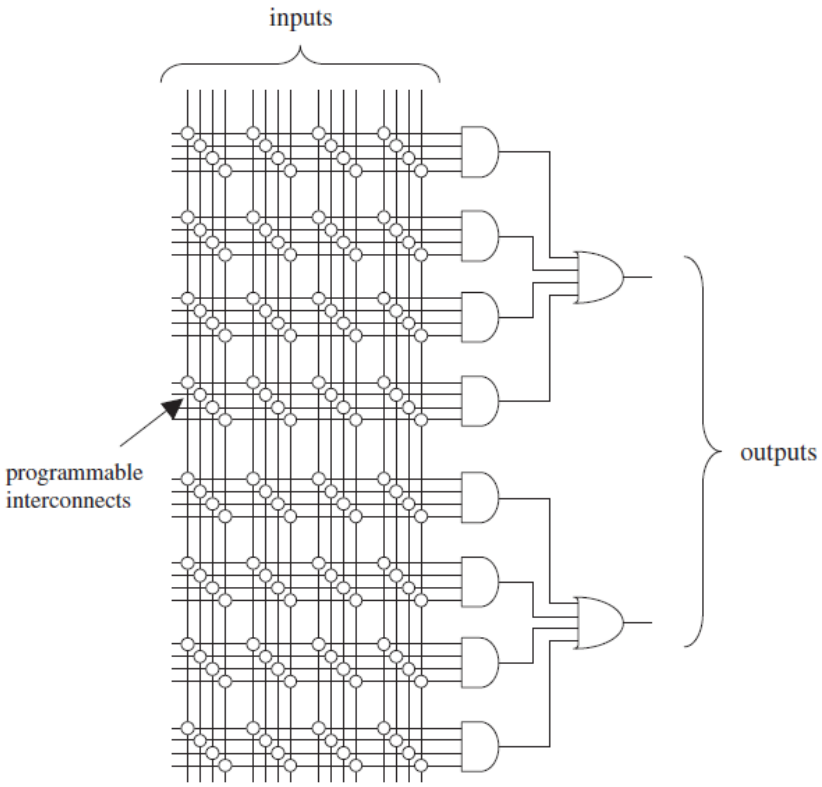
---

[15] Arithmetic/Logical Unit
[16] Application-Specific Integrated Circuit
[17] Small Scale Integration
[18] Simple Programmable Logic
[19] Complex Programmable Logic

۲-۳-۱. تراشه‌های SPLD

این نوع از تراشه در ابتدا به منظور پیاده‌سازی گیت‌های منطقی به وجود آمدند. لذا تنها در پیاده‌سازی مدارهای ترکیبی کاربرد داشتند. به منظور حل این مشکل، به هر یک از خروجی‌های این تراشه‌ها یک فلیپ فلاپ اضافه و امکان پیاده‌سازی مدارهای ترتیبی نیز میسر شد. این تراشه‌ها به سه نوع PAL[20]، PLA[21] و GAL[۲۲] تقسیم می‌شوند که تفاوتشان در معماری سخت‌افزاری آن‌هاست. در ادامه، ساختار داخلی این سه نوع قطعه را بررسی خواهیم کرد.

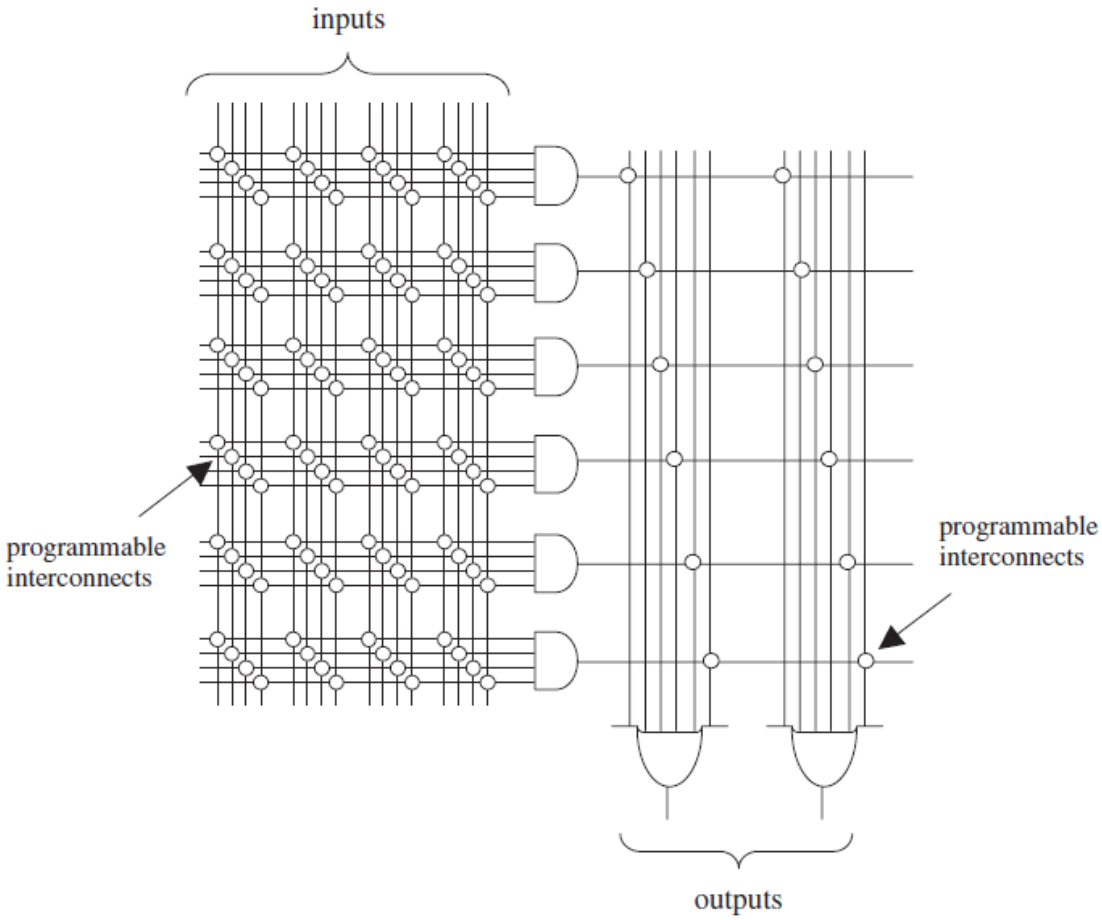
تصویر ۲-۱ ساختار داخلی یک PAL را نشان می‌دهد. با توجه به این تصویر، ساختار داخلی این قطعات شامل گیت‌های منطقی برنامه‌پذیر AND و گیت‌های ثابت OR می‌شود. در این حالت، به کمک مینترم‌های موجود در جدول صحت و نمایش یک مدار ترکیبی به فرم مجموع حاصل‌ضرب‌ها[۲۳]، این دسته از مدارها قابل پیاده‌سازی هستند.



تصویر ۲-۱ – ساختار PAL [۹]

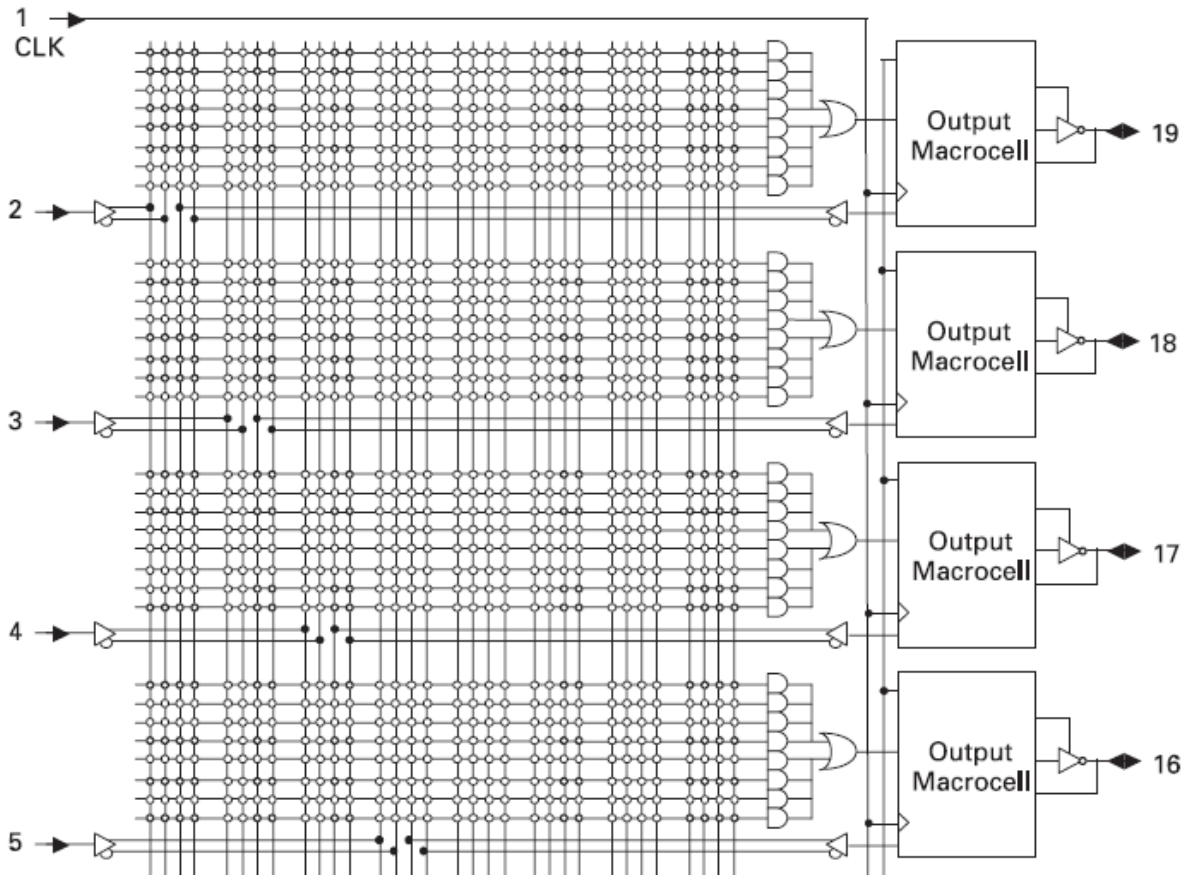
[20] Programmable Array Logic
[21] Programmable Logic Array
[22] Generic PAL
[23] Sum of Product (SOP)

تصویر ۲-۲ ساختار داخلی یک PLA را نشان می‌دهد. با توجه به این تصویر، ساختار داخلی این قطعات مشابه قطعات PAL هستند، البته، با این تفاوت اساسی که علاوه‌بر برنامه‌پذیر بودن گیت‌های AND، گیت‌های OR نیز قابلیت برنامه‌پذیری دارند. مهم‌ترین مزیت این تغییر، انعطاف‌پذیری بیشتر در پیاده‌سازی است. با این‌حال، به علت بزرگتر بودن ثابت زمانی گره‌های داخلی، سرعت این قطعات به PAL کمتر است.



تصویر ۲-۲ – ساختار PLA [۹]

تصویر ۲-۳ ساختار داخلی یک GAL را نشان می‌دهد. در این قطعات، نسبت به یک PAL، چند تغییر اساسی رخ داده است. اولین تغییر مربوط به اضافه شدن سلول Microcell به ساختار است. این سلول‌ها شامل چند گیت منطقی، مالتی پلکسر و فیلپ فلاپ هستند. لذا امکان طراحی مدارهای ترتیبی نیز فراهم شده است. دومین تغییر مهم، برنامه‌پذیر بودن این سلول‌هاست. و آخرین تغییر مهم، اضافه شدن یک فیدبک از خروجی این سلول‌ها به ورودی است. در نتیجه، علاوه‌بر وجود فلیپ فلاپ و طراحی مدارهای ترتیبی، امکان استفاده از فیدبک خروجی نیز وجود دارد.



تصویر ۲-۳ – بخشی از ساختار GAL [۹]

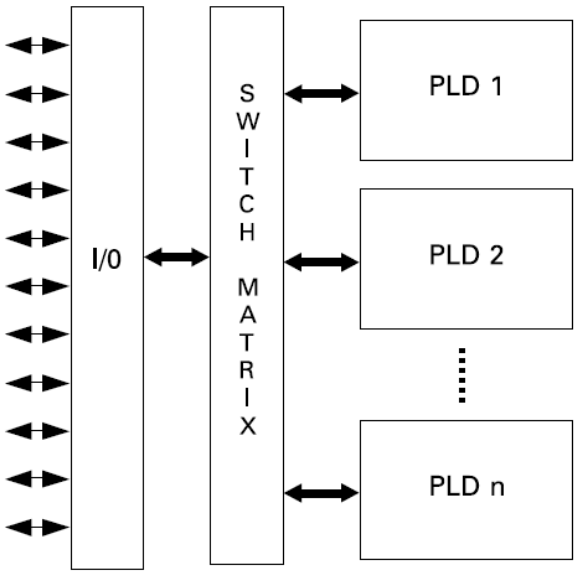
۲-۳-۲. تراشه‌های CPLD

تصویر ۲-٤ ساختار داخلی یک CPLD را نشان می‌دهد. این قطعات شامل کلیدهای برنامه‌پذیر(Switch matrix)، ورودی/خروجی‌های قابل تنظیم و تعداد مشخصی PLD (عموما قطعات GAL) هستند. علاوه‌بر این ساختار، به طور معمول، این تراشه‌ها امکانات محدود دیگری را مانند JTAG و پشتیبانی از سایر ولتاژهای استاندارد منطقی (۱/۸، ۲/۵ و ۵) شامل می‌شوند.

۲-۳-۳. تراشه‌های FPGA

FPGA نیز نوعی دیگر از مدار مجتمع برنامه‌پذیر به منظور پیاده‌سازی مدارهای دیجیتال است. با توجه به تصویر ۲-۵، ساختار این تراشه از سه بخش مهم بلوک منطقی برنامه‌پذیر (CLB[24])، شبکه‌های اتصال قابل تنظیم و بلوک‌های ورودی/خروجی (IO) تشکیل شده است. تراشه‌های پیشرفته‌تر دیگر نیز ممکن است دارای بخش‌های دیگری مانند واحدهای DSP، Block RAM و DCM باشند. مهمترین المان FPGA که امکان ساخت مدارهای منطقی به صورت ترتیبی و ترکیبی را فراهم می‌کند، CLB است. با توجه به تصویر ۲-٦-الف، هر CLB از دو

[24] Configurable Logic Block



تصویر ۲-٤ – بخشی از ساختار GAL [۹]

قسمت مهم زیر تشکیل شده است:

- **LUT[25]: یک حافظه‌ی قابل برنامه‌ریزی است که دارای K بیت آدرس ورودی و 2^K سلول حافظه است و به کمک آن، امکان ساخت یک مدار ترکیبی به کمک مینترم‌هایش فراهم می‌شود. به عنوان نمونه، تصویر ۲-٦-ب ساختار یک LUT با سه ورودی A_0، A_1 و A_2 را نشان می‌دهد. با توجه به این تصویر، این حافظه دارای هشت سلول حافظه است که در صورت ۱ شدن مقدار هر کدام، مینترم متناظر آن فعال شده و امکان ساخت یک مدار ترکیبی فراهم می‌شود.**
- **FF/LATCH: یک فلیپ-فلاپ یا لچ قابل تنظیم است که به صورت‌های سنکرون یا آسکرون برنامه‌ریزی شده و امکان ساخت مدار ترتیبی را فراهم می‌کند.**

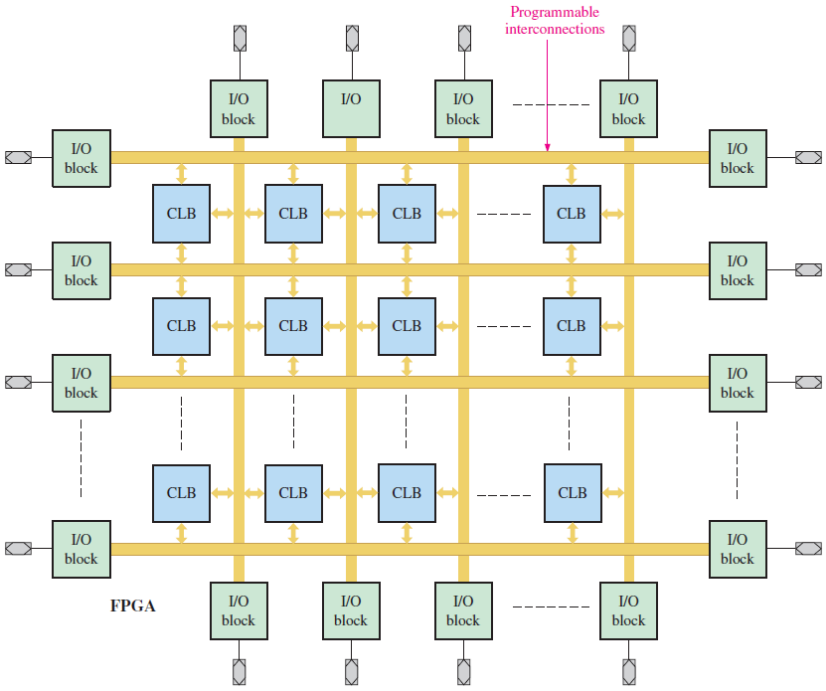
در بخش ۲-۵ و ۲-٦، به طور خاص در ارتباط با تراشه‌های Spartan 6 LX9، محصول شرکت Xilin و پیاده‌سازی مدارهای دیجیتال بر روی آن‌ها، صحبت خواهد شد.

۲-٤. تراشه‌های ASIC

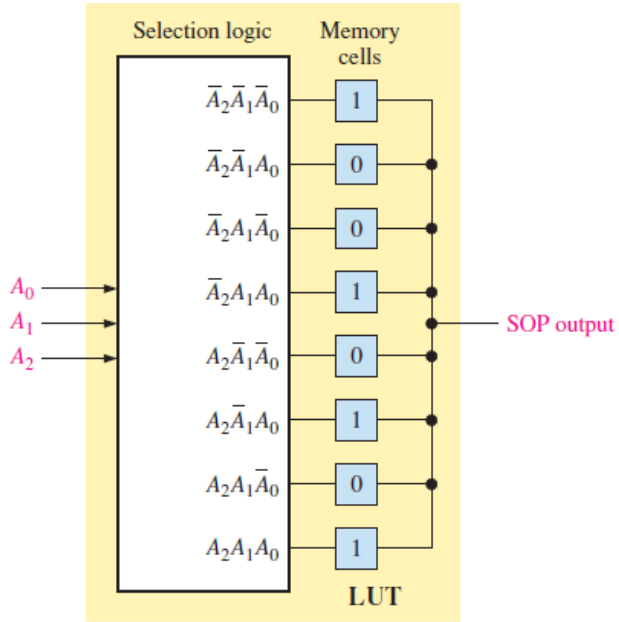
تراشه‌های ASIC به منظور انجام یک هدف خاص، به عنوان نمونه کاربردهای شبکه، استفاده در تلفن‌های همراه و کاربردهای گرافیکی طراحی شده‌اند. طراحان این تراشه‌ها، به کمک ترانزیستورها، گیت‌های منطقی

[25] Look-up Table

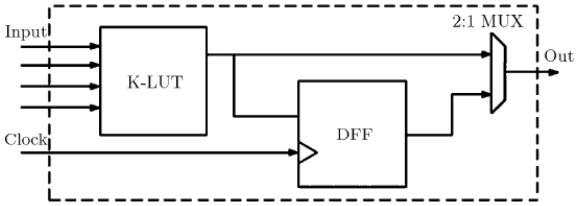
می‌سازند و سپس این گیت‌ها را در کنار یکدیگر چینش می‌کنند. به همین علت، قطعات ASIC دارای مساحت کمتر و سرعت بیشتری نسبت به تراشه‌های FPGA هستند. با این‌حال، فرآیند ساخت تراشه‌های ASIC گران قیمت و زمان‌بر است و تنها برای کاربردهای خاص و تولیدهای انبوه به صرفه هستند.



تصویر ۲-۵ – ساختار کلی یک تراشه‌ی FPGA [۷]



ب – ساختار یک LUT با ۳ بیت ورودی [۷]



الف – ساختار CLB [۸]

تصویر ۲-۶ – المان‌های مهم یک CLB

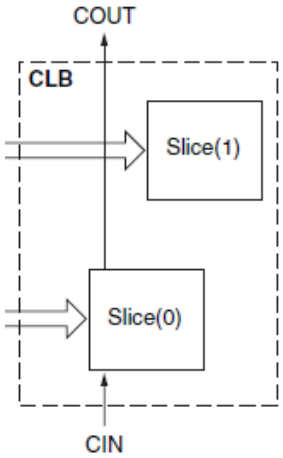
۲-۵. بررسی تراشه‌ی Spartan 6 LX9 و معماری سخت‌افزاری آن

Spartan 6 LX9 یک تراشه‌ی FPGA از خانواده‌ی اسپارتان ٦[۲۶] و محصول شرکت زایلینکس[۲۷] است. در حالت کلی، این خانواده شامل سیزده مدل تراشه‌ی FPGA است که مشخصات منابع سخت‌افزاری آن‌ها در جدول ۲-۱ قابل مشاهده است. به عنوان نمونه، تراشه‌ی Spartan 6 LX9 شامل ۱۱٤٤۰ فلیپ فلاپ، ٦۰۰ واحد اسلایس[۲۸]، ۱٦ واحد DSP[29]، ۳۲ عدد رم 18Kb و ۲۰۰ عدد ورودی/خروجی قابل تنظیم است. با توجه به تصویر ۲-۷، در این خانواده از تراشه‌های FPGA شرکت زایلنکس، هر واحد CLB شامل دو واحد اسلایس می‌شود که هر کدامشان دارای چهار واحد LUT و هشت فلیپ فلاپ است. لذا در حالت مجموع، هر CLB شامل هشت LUT و شانزده فلیپ فلاپ است. تصویر ۲-۸ ساختار داخلی یک اسلایس را نشان می‌دهد. جزئیات مربوط به این ساختار و روش کار آن به دانشجو سپرده می‌شود.

جدول ۲-۱ – منابع سخت‌افزاری خانواده‌ی Spartan 6 [٤]

Device	Logic Cells(1)	Configurable Logic Blocks (CLBs)			DSP48A1 Slices(3)	Block RAM Blocks		CMTs(5)	Memory Controller Blocks (Max)(6)	Endpoint Blocks for PCI Express	Maximum GTP Transceivers	Total I/O Banks	Max User I/O
		Slices(2)	Flip-Flops	Max Distributed RAM (Kb)		18 Kb(4)	Max (Kb)						
XC6SLX4	3,840	600	4,800	75	8	12	216	2	0	0	0	4	132
XC6SLX9	9,152	1,430	11,440	90	16	32	576	2	2	0	0	4	200
XC6SLX16	14,579	2,278	18,224	136	32	32	576	2	2	0	0	4	232
XC6SLX25	24,051	3,758	30,064	229	38	52	936	2	2	0	0	4	266
XC6SLX45	43,661	6,822	54,576	401	58	116	2,088	4	2	0	0	4	358
XC6SLX75	74,637	11,662	93,296	692	132	172	3,096	6	4	0	0	6	408
XC6SLX100	101,261	15,822	126,576	976	180	268	4,824	6	4	0	0	6	480
XC6SLX150	147,443	23,038	184,304	1,355	180	268	4,824	6	4	0	0	6	576
XC6SLX25T	24,051	3,758	30,064	229	38	52	936	2	2	1	2	4	250
XC6SLX45T	43,661	6,822	54,576	401	58	116	2,088	4	2	1	4	4	296
XC6SLX75T	74,637	11,662	93,296	692	132	172	3,096	6	4	1	8	6	348
XC6SLX100T	101,261	15,822	126,576	976	180	268	4,824	6	4	1	8	6	498
XC6SLX150T	147,443	23,038	184,304	1,355	180	268	4,824	6	4	1	8	6	540

[26] Spartan 6
[27] Xilinx
[28] Slice
[29] Digital Signal Processing



Slices	LUTs	Flip-Flops
2	8	16

تصویر ۲-۷ – ساختار CLB در یک تراشه‌ی Spartan 6 [٤]

تمرین

دیتاشیت تراشه‌های Spartan 6 را مطالعه کرده و به سولات زیر پاسخ دهید [مراجع ۳-٦]

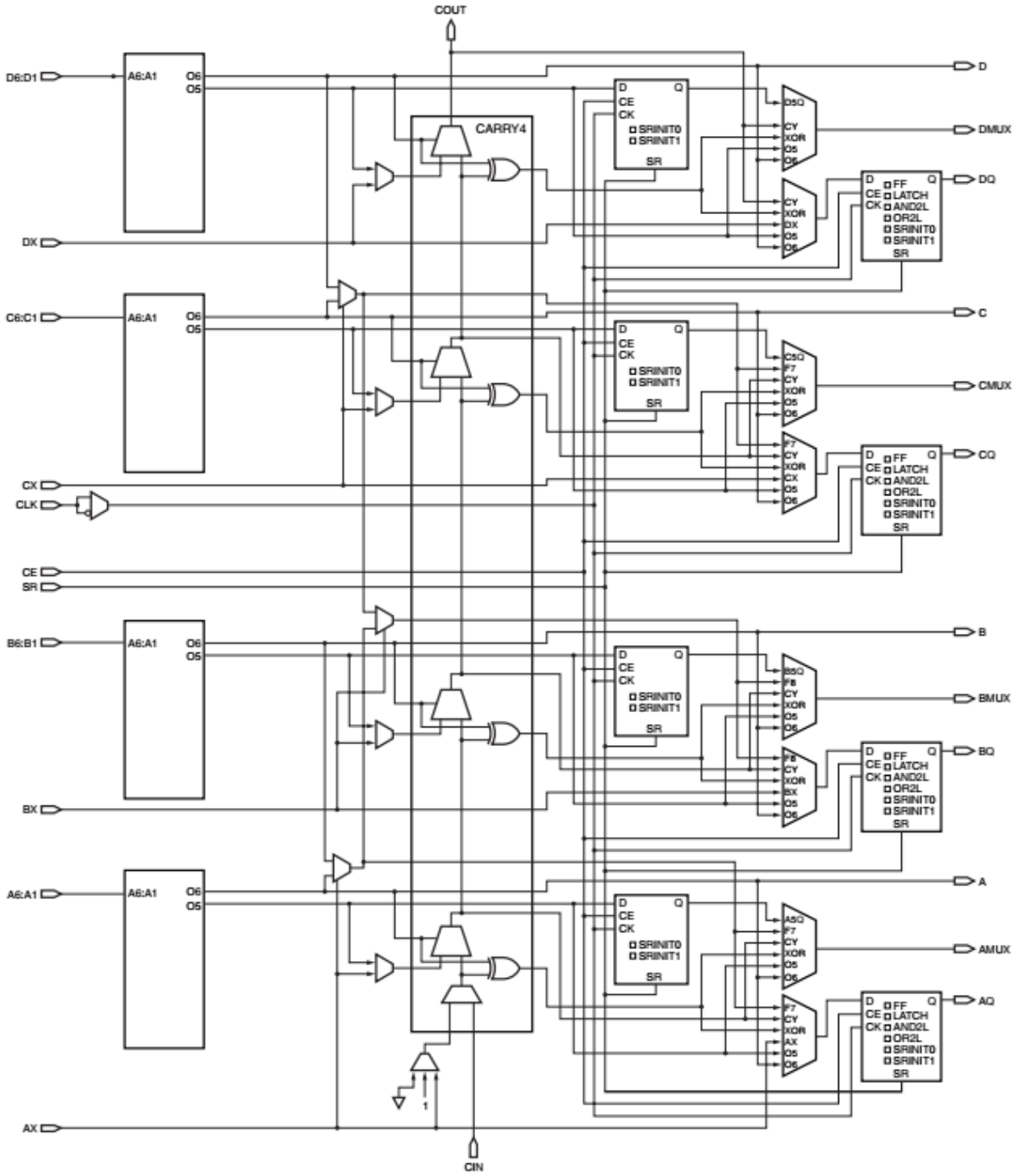
۱) تراشه‌ی FPGA این شرکت دارای چه امکاناتی است و از چه بخش‌هایی تشکیل شده (به صورت تیتروار توضیح دهید).

۲) تفاوت و نوع کاربرد واحدهای SLICEM، SLICEL و SLICEX را بیان کنید.

۳) واحد DCM به چه منظور استفاده می‌شود؟

٤) واحد DSP-48 به چه منظور استفاده می‌شود و حداکثر فرکانس عملکرد آن چقدر است؟

۵) در مورد ساختار Block RAMهای تراشه‌ی Spartan 6 توضیح دهید.



تصویر ۲-۸ - ساختار داخلی یک اسلایس [٤]

بخش دوم – آشنایی با نرم‌افزار Xilinx ISE 14.7

هدف این بخش آشنایی با نرم‌افزار ISE است. بدین منظور، ابتدا در آزمایش ۱، با ساختار کلی این نرم‌افزار آشنا می‌شویم و به کمک ابزار شماتیک آن، یک مدار منطقی را طراحی خواهیم کرد. سپس به کمک نرم‌افزار ISIM، مدار طراحی شده را در سطح رفتاری شبیه‌سازی خواهیم کرد. در ادامه سای سنتز در آزمایش سوم، به روال پیاده‌سازی، پیکربندی و شبیه‌سازی در سطح Post-Place and Route خواهیم پرداخت. و در انتهای این بخش، در آزمایش چهارم، یک کد VHDL ایجاد کرده و تمامی موارد بررسی شده در آزمایش‌های یک تا سه را مجددا انجام خواهیم داد.

آزمایش شماره‌ی ۱

هدف:

- آشنایی با برد توسعه‌ی Posedge و مستندات آن
- آشنایی با نرم‌افزار Xilinx ISE 14.7 و ابزار طراحی شماتیک
- آشنایی با روال طراحی یک مدار دیجیتال با مثال طراحی یک تمام جمع‌کننده

تئوری آزمایش:

در این آزمایش، به ترتیب سه مرحله را طی خواهیم کرد. در مرحله‌ی اول، با بررسی مستندات برد توسعه‌ی پازج، این پلتفرم را بررسی می‌کنیم و امکانات آن را مورد بررسی قرار می‌دهیم. در مرحله‌ی بعد، محیط نرم‌افزار ISE و آموزش ساخت یک پروژه در ابزار شماتیک معرفی می‌شود. در انتها، به کمک مثال طراحی تمام جمع‌کننده، روال طراحی مدارهای دیجیتال بررسی می‌شود.

آزمایش:

قدم ۱ – بررسی مستندات برد توسعه‌ی پازج

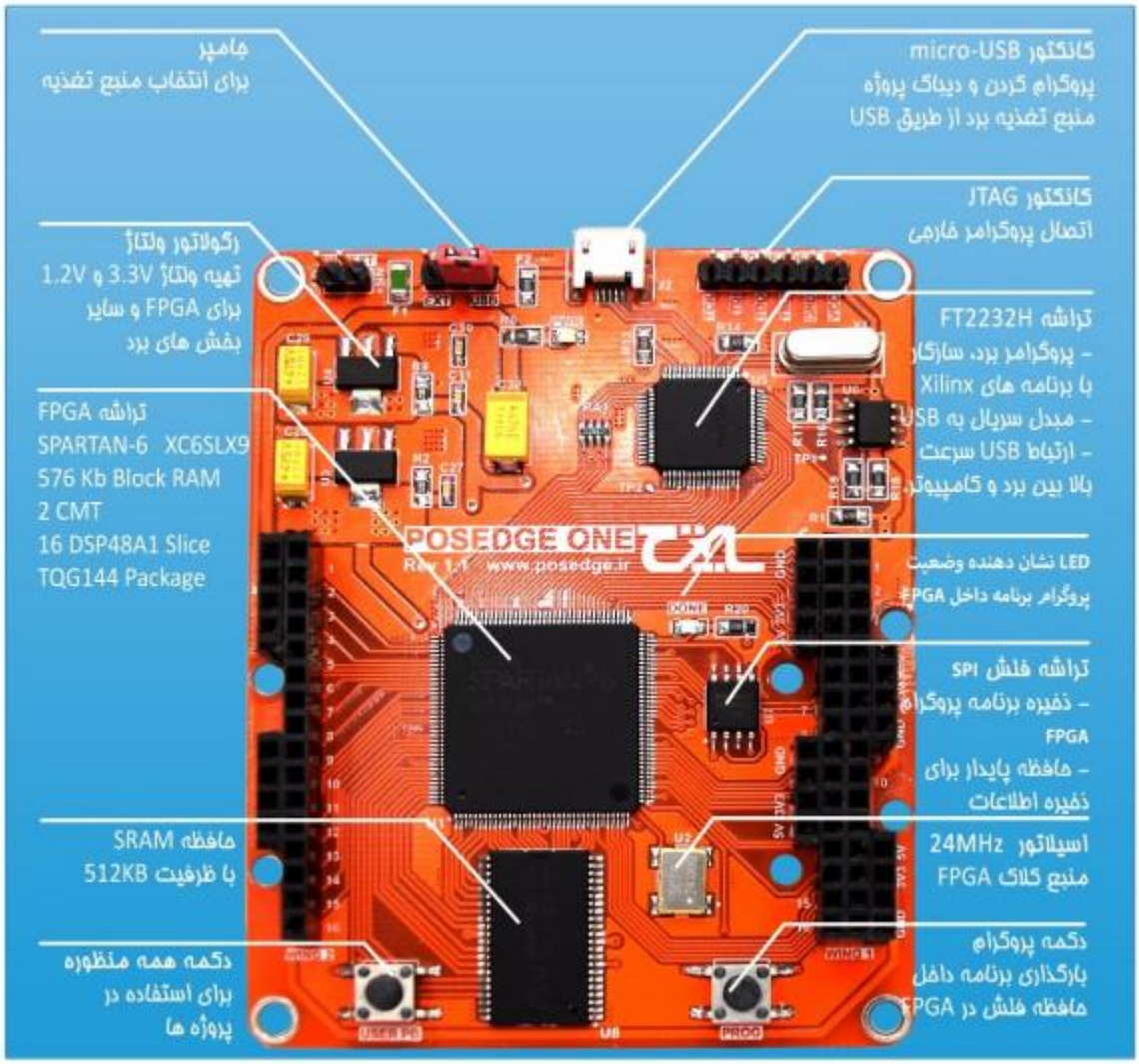
این برد بر مبنای تراشه‌ی Spartan6-LX9 بنا شده است و به کمک درگاه USB برنامه‌دهی می‌شود. تصویر ۱ و ۲ به صورت خلاصه ساختار این برد توسعه را نشان می‌دهد. به طور خلاصه این امکانات در این پلتفرم آموزشی قرار گرفته است:

- تراشه‌ی Spartan6-LX9
- پروگرامر USB با قابلیت ارتباط با نرم‌افزار ISE
- حافظه‌ی Flash جهت ذخیره‌ی برنامه
- ٤٨ پین ورودی/خروجی
- بال MegaWing جهت فراهم آوردن امکانات آموزشی، مانند سون‌سگمنت، LED، درگاه VGA و SD-card

قدم ۲ – آشنایی با نرم‌افزار Xilinx ISE 14.7 و محیط طراحی شماتیک

این نرم‌افزار شامل چند زیربرنامه است که اعمال طراحی مدارهای دیجیتال را، به صورت شماتیک یا کدنویسی HDL، و سپس پیکربندی آن‌ها را بر روی بردهای FPGA انجام می‌دهند. به منظور آشنایی با این نرم‌افزار و استفاده از ابزار شماتیک آن، مراحل زیر را طی کنید:

۱. پس از نصب نرم‌افزار ISE، برنامه‌ی Project Navigator را باز کنید.



تصویر ۱ – ساختار برد توسعه‌ی پازج

۲. مطابق تصویر ۳، به کمک یکی از دو روش نشان داده شده، بر روی New Project کلیک کنید و یک پروژه‌ی جدید بسازید.

۳. بعد از کلیک بر روی Next، مطابق تصویر ٤، ابتدا نام پروژه (Adder) و محل ذخیره‌سازی آن را مشخص کنید و Type of top-level source را بر روی Schematic قرار دهید. دقت داشته باشید که نام‌گذاری پروژه از قاعده‌ی خاصی پیروی می‌کند که به صورت زیر است:

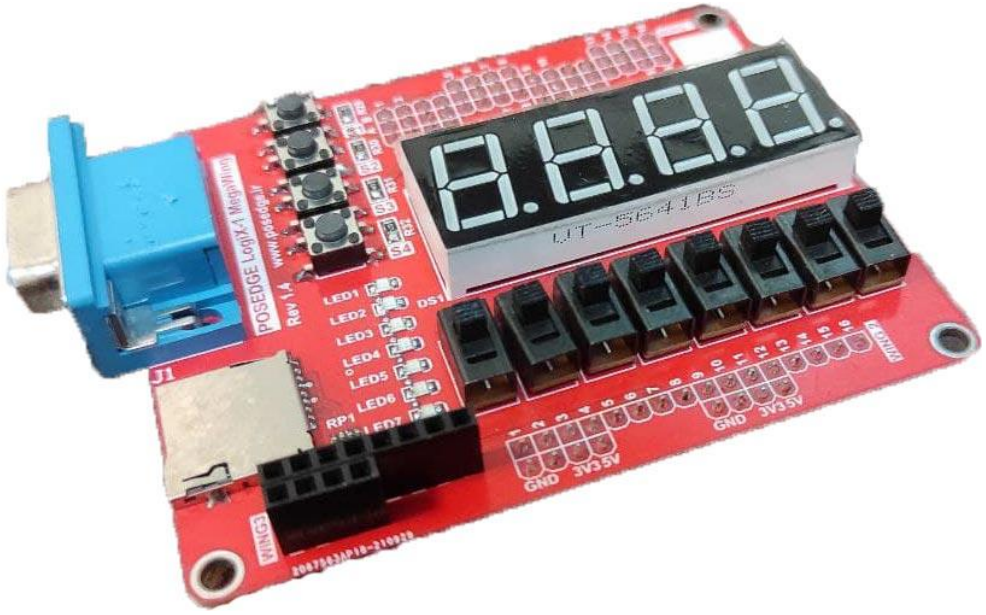
i. تنها استفاده از حروف انگلیسی بزرگ یا کوچک، اعداد انگلیسی و کارکتر underscore یا "_" (Shift + -) مجاز است (استفاده از Space و سایر کارکترها مجاز نیست).

ii. نام هر بخش **حتما باید** با حروف انگلیسی شروع شود

iii. نام هیچ بخشی نمی‌تواند با underscore تمام شود

iv. دو underscore نمی‌تواند به صورت پیاپی در کنار هم قرار بگیرد

v. ترجیحا نام‌هایی استفاده شود که نشان‌دهنده‌ی آن شناسه باشند. مثلا FA برای تمام‌جمع‌کننده (Full Adder) یا MUX برای مالتی‌پلکسر. البته رعایت نکردن این بخش باعث ایجاد خطا در نرم‌افزار نخواهد شد و تنها برای خوانایی بهتر کد یا طراحی نیاز است.



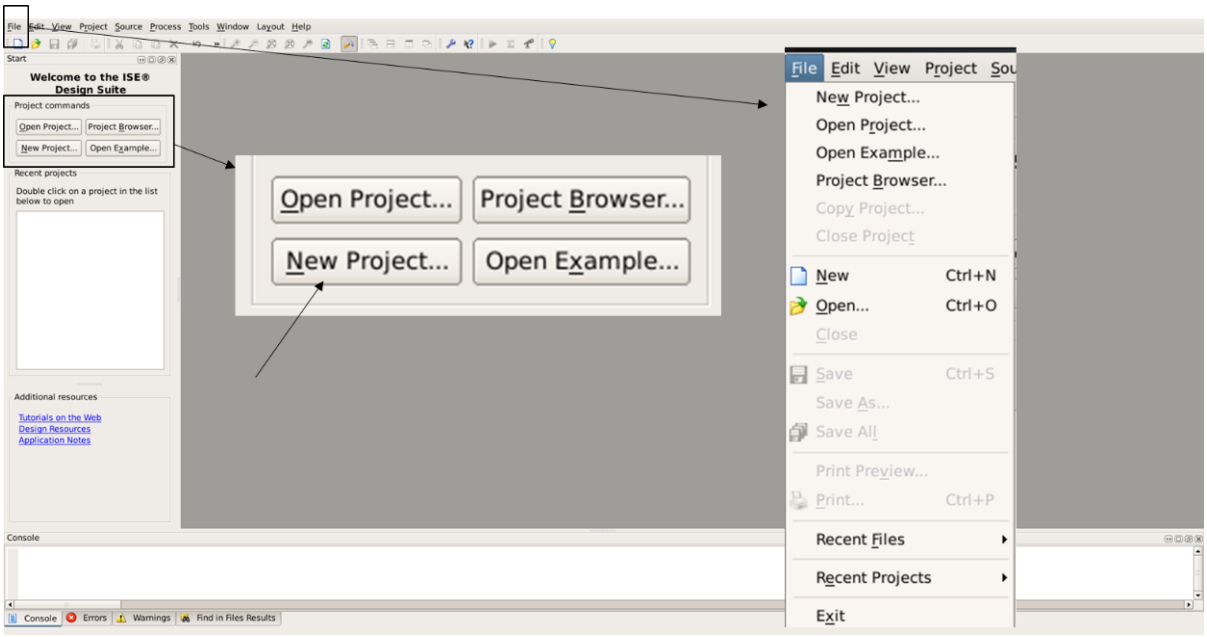
تصویر ۲ – ساختار بال MegaWing

۴. سپس، بعد از کلیک بر روی Next، با توجه به تصویر ۵، نوع تراشه و تنظیمات را با دقت اعمال کنید.

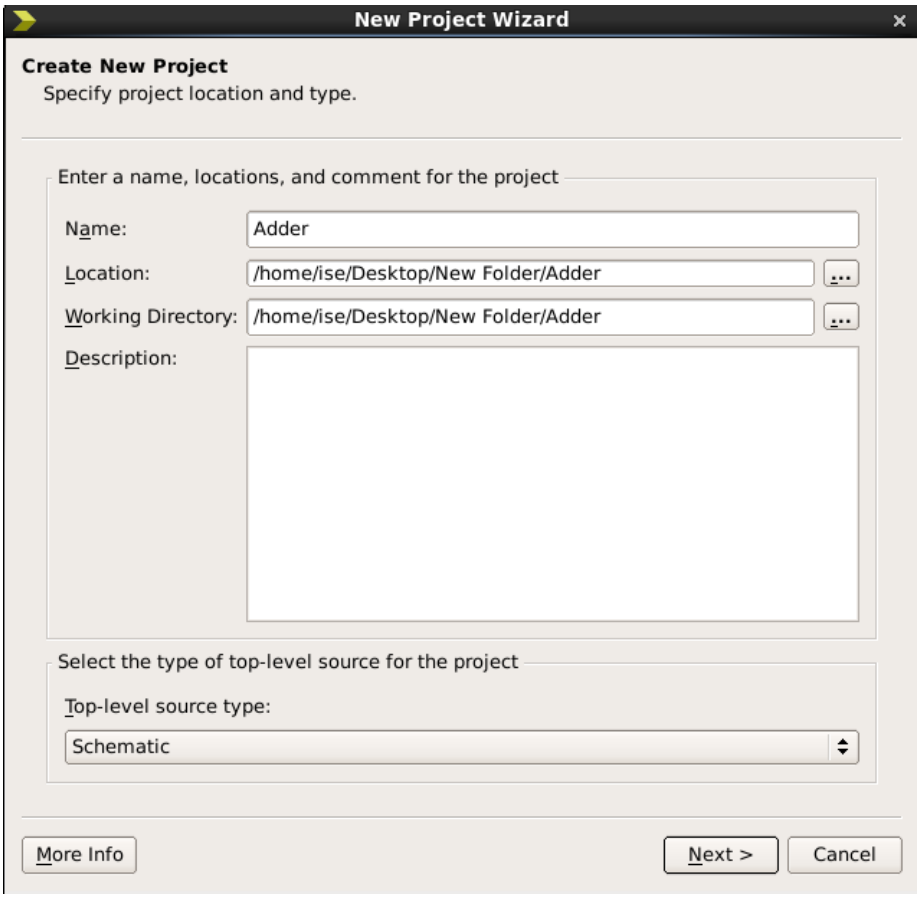
۵. بعد از کلیک بر روی Next، در پنجره‌ی آخر تنظیمات و موارد مهم مربوط به پروژه را چک کنید و در صورت عدم وجود اشکال، بر روی Finish کلیک کنید.

۶. پس از ساخت پروژه، محیط دیگر نرم‌افزار در تصویر ٦ نشان داده می‌شود. چند بخش مهم این تصویر عبارت‌اند از:

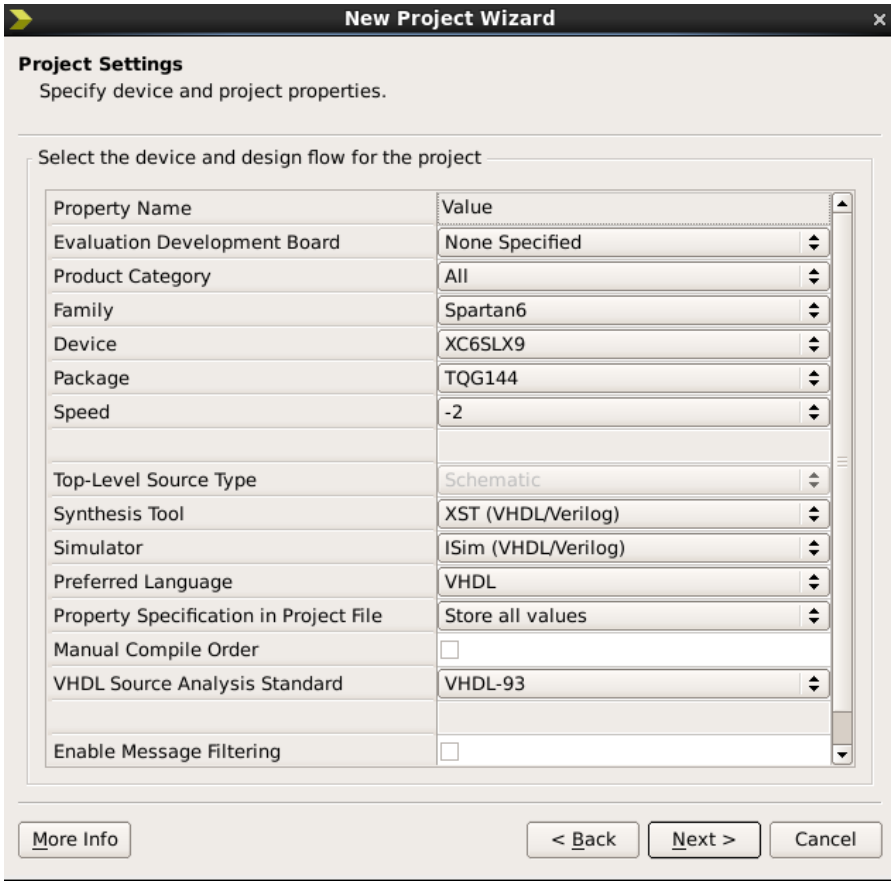
۱. View: به کمک این بخش می‌توان کدها و فایل‌های شماتیک را در دو بخش شبیه‌سازی و پیاده‌سازی تحلیل کرد.



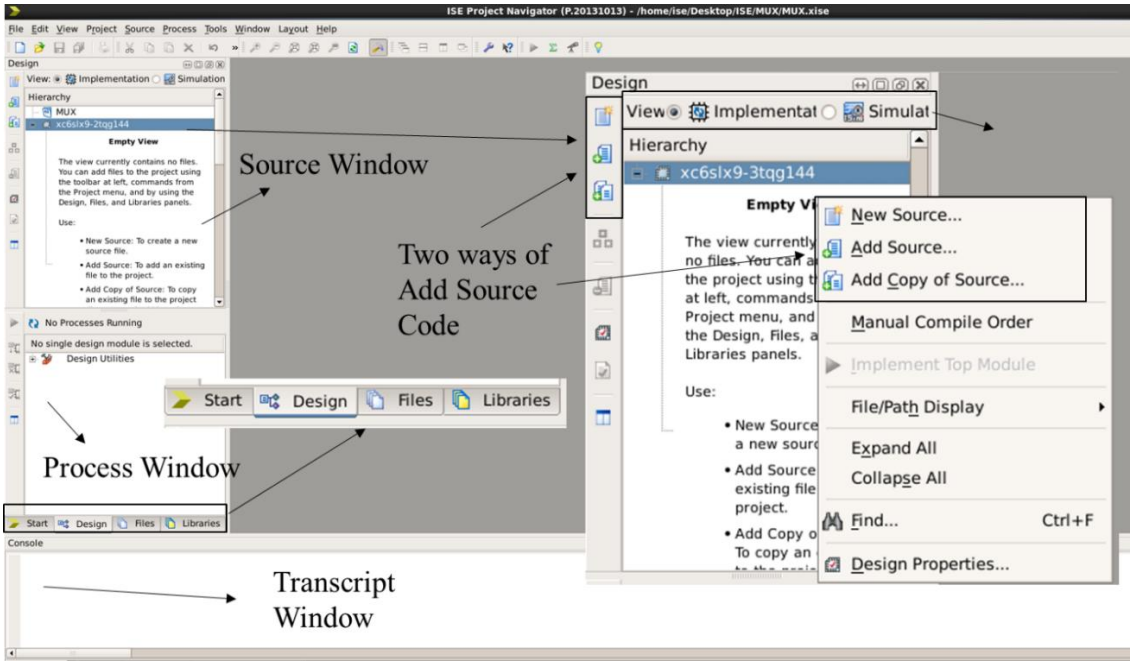
تصویر ۳ – ساخت پروژه‌ی جدید در نرم‌افزار ISE



تصویر ٤ – اعمال تنظیمات بر روی پروژه‌ی ساخته شده



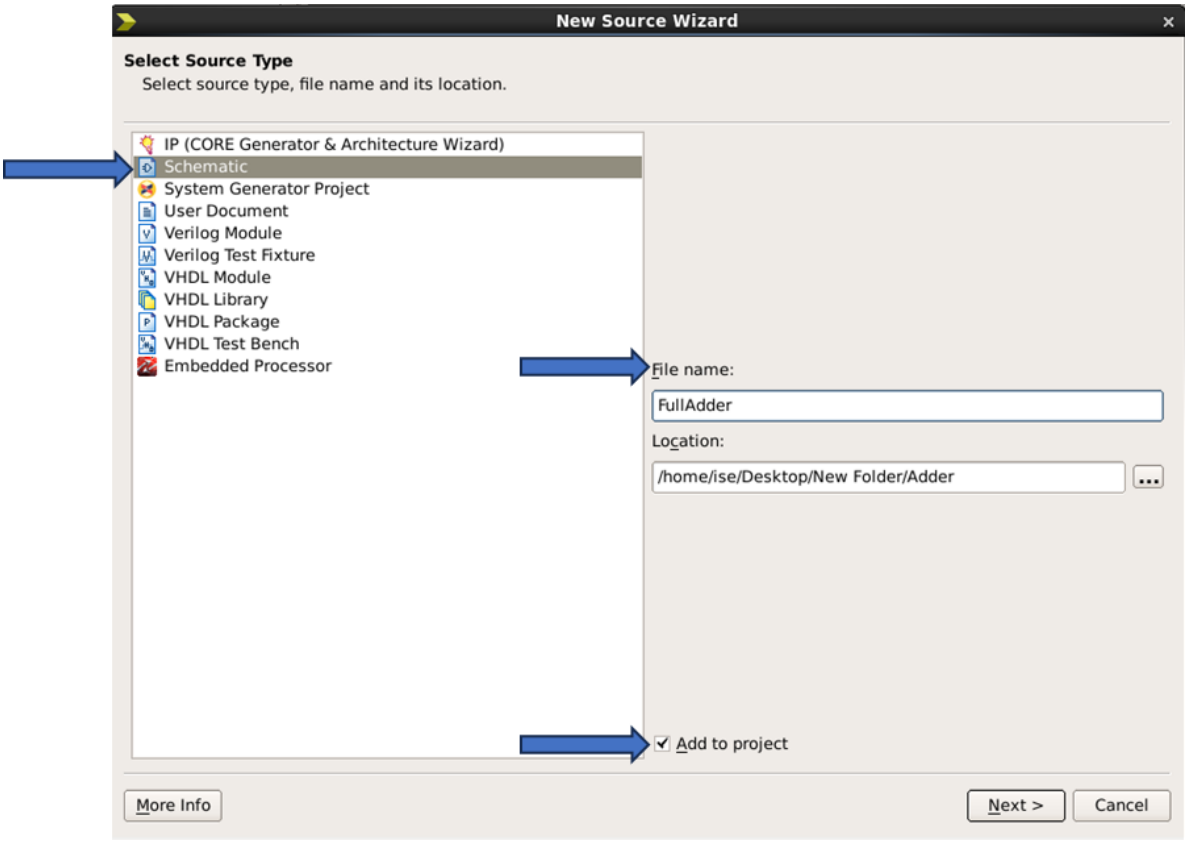
تصویر ۵ – اعمال تنظیمات بر روی پروژه‌ی ساخته شده



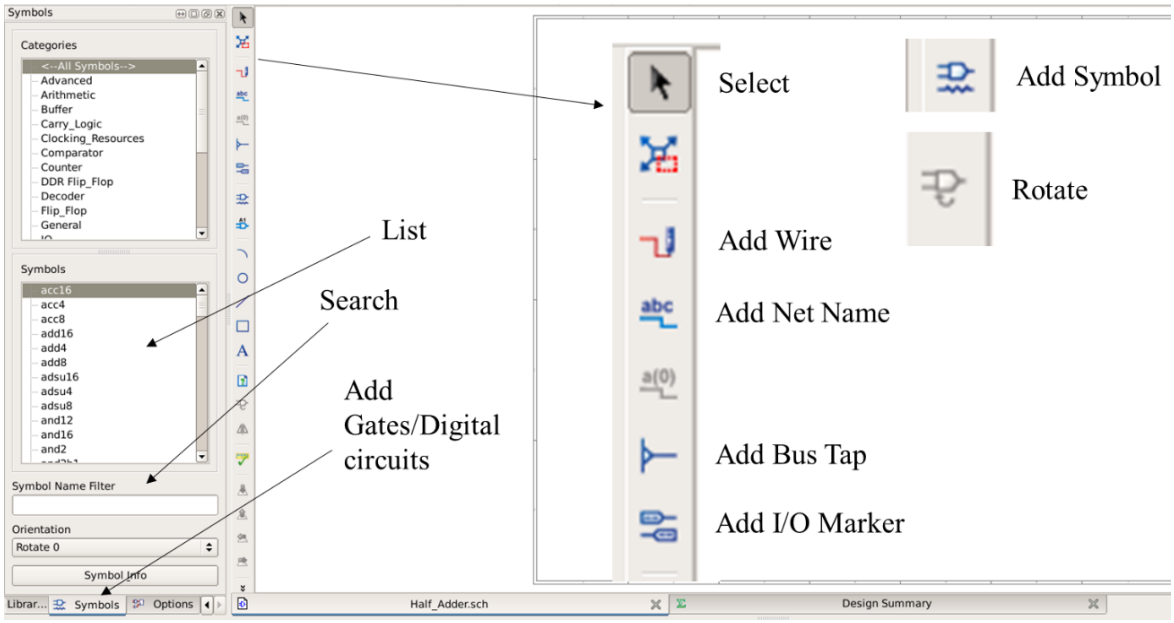
تصویر ٦– پنجره‌های مهم در نرم‌افزار ISE و روش ساخت یک فایل شماتیک

۲. Source Window: ساختار درختی کدهای زبان سخت‌افزار، فایل‌های شماتیک و سایر موارد مهم در این بخش نشان داده می‌شود. همچنین در این بخش می‌توان یک فایل جدید به پروژه اضافه کرد.
۳. Process Window: فرآیند مربوط به پیاده‌سازی و شبیه‌سازی مدارها در این قسمت انجام می‌شود. همچنین در قسمت پایین این بخش، می‌توان بین سایر پنل‌های نرم‌افزار، یعنی start، design، library و file، جابه‌جا شد.
۴. Transcript Window: پیام‌های مربوط به فرآیندهای مهم در این قسمت قابل مشاهده است.
۷. با توجه به تصویر ٦، با یکی از دو روش نشان داده شده، بر روی New Source کلیک کنید.
۸. با توجه به تصویر ۷، در پنل سمت چپ حتما گزینه‌ی Schematic را انتخاب کرده و در بخش پایین نام ماژول، حتما تیک Add to project فعال باشد. سپس برای پروژه‌ی خود نام (FullAdder) را انتخاب کرده و بر روی Next کلیک کنید.
۹. در آخرین پنجره نیز بر روی Finish کلیک کنید تا فایل شماتیک ساخته شود.
۱۰. بعد از انجام فوق، تصویر ۸ نمایش داده می‌شود. همانگونه که ملاحظه می‌شود، پنل Symbol در کنار سایر پنل‌های start، design و library اضافه شده است. از این پنل به منظور طراحی شماتیک استفاده می‌شود. ابزارهای مهم طراحی شماتیک که در این تصویر قابل مشاهده هستند عبارت‌اند از:

a. بخش Symbol: شامل المان‌های مداری به همراه دسته‌بندی آن‌ها و اعمال فیلتر به منظور جستوجوی بهتر
b. ابزار Select: انتخاب یک المان به منظور اعمال تغییرات و جابه‌جایی
c. ابزار Add Wire: ترسیم سیم و اتصالات داخلی یک مدار
d. ابزار Add Net Name: نام‌گذاری یک سیم یا اتصال
e. ابزار Add Bus Tap: ابزار شاخه‌بندی یک باس داده
f. ابزار Add I/O Marker: اضافه کردن پورت ورودی/خروجی
g. ابزار Add Symbol: افزودن یک المان مداری
h. ابزار Rotate: به منظور چرخش یک المان استفاده می‌شود



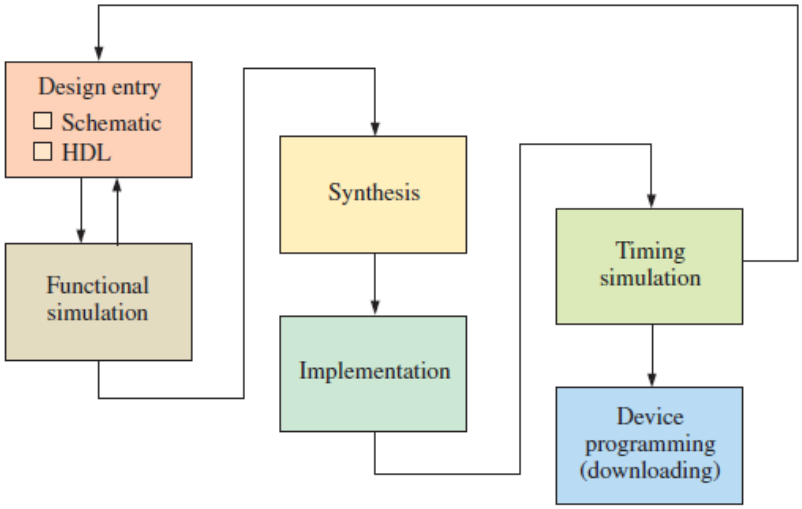
تصویر ۷ – روش ساخت یک فایل شماتیک



تصویر ۸ –آشنایی با ابزارهای مهم طراحی شماتیک

قدم ۳ – روال طراحی مدارهای دیجیتال در نرم‌افزار با مثال تمام جمع‌کننده

تصویر ۹ روند طراحی مدارهای دیجیتال تا پیکربندی آن‌ها را بر روی بردهای FPGA به صورت خلاصه نشان داده می‌دهد. این مراحل در شش بخش انجام می‌شوند. در ادامه هر یک از این بخش‌ها بررسی می‌شوند.

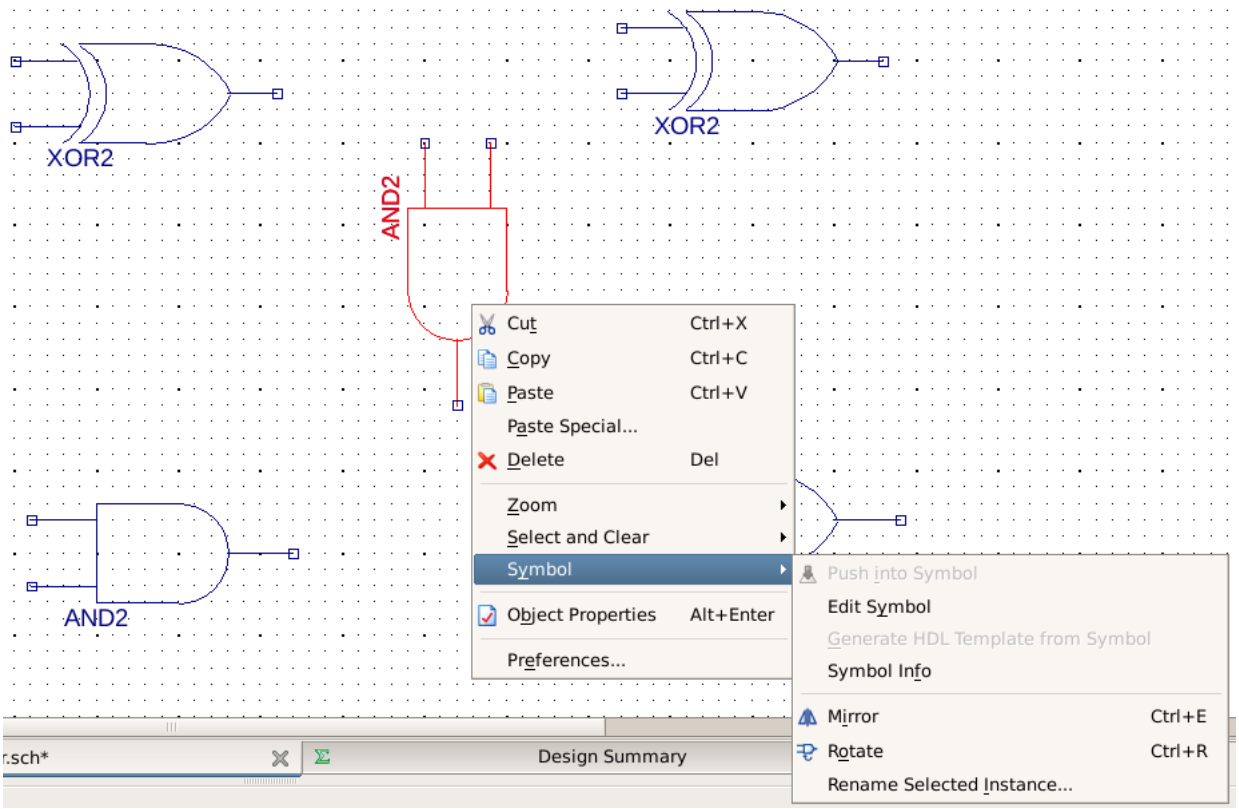


تصویر ۹ – روال پیاده‌سازی یک مدار دیجیتال بر روی FPGA [۹]

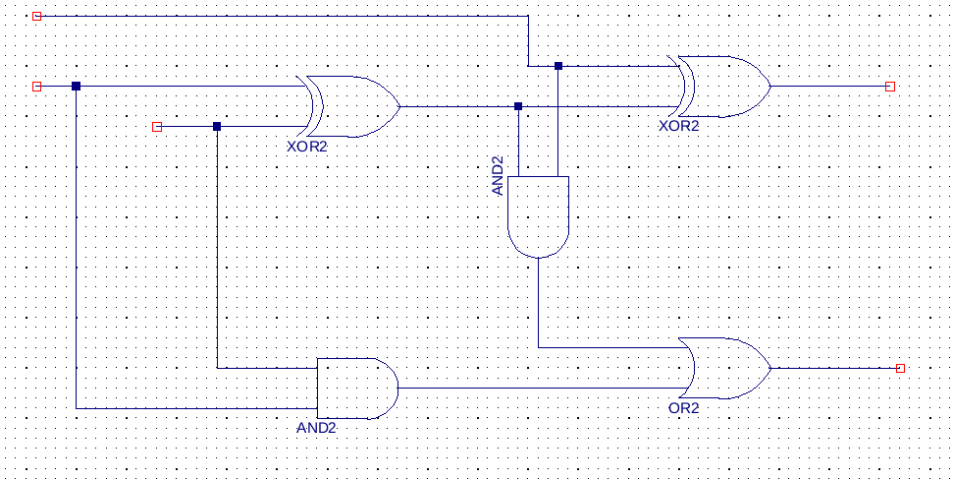
بخش ۱- طراحی شماتیک

در بخش قبل، یک فایل شماتیک ساخته شد. در این بخش، مدار مورد نظر به صورت شماتیک طراحی می‌شود. بدین منظور، مراحل زیر را انجام دهید:

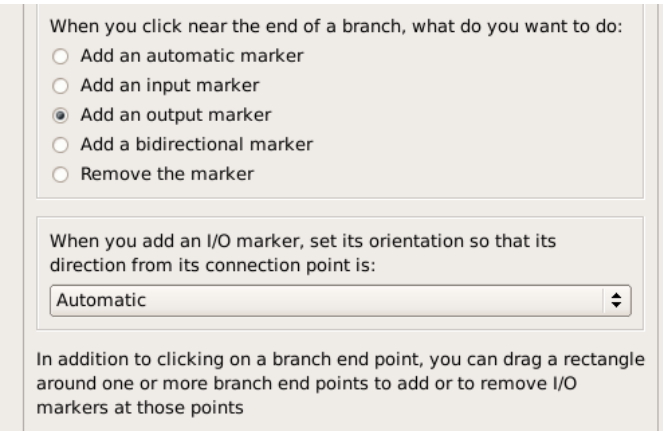
۱. در پنجره‌ی Symbol در تصویر ۸، المان‌های xor2، and2 و or2 را یافته و به صورت تصویر ۱۰ در کنار هم قرار بدهید. دقت کنید که برای چرخش یک المان، مطابق تصویر ۱۰، بر روی آن راست کلیک کرده و در بخش Symbol، بر روی Rotate کلیک کنید.
۲. به کمک ابزار Add Wire (در تصویر ۸ نشان داده شده است) مدار تصویر ۱۱ را ترسیم کنید.
۳. به کمک ابزار Add I/O Marker، یک پورت ایجاد کنید. مطابق تصویر ۱۲، می‌توانید تنظیم کنید که پورت ورودی یا خروجی باشد. سپس تصویر ۱۳ را بسازید. جهت تغییر نام هر پورت، پس ایجاد آن بر روی آن دبل کلیک کرده و به مشابه تصویر ۱٤، نام آن را تغییر بدهید.
۴. پس از نهایی کردن مراحل فوق، در بخش Menu بر روی File رفته و بر روی Save کلیک کنید (همچنین می‌توانید بر روی آیکون Save کلیک کنید).



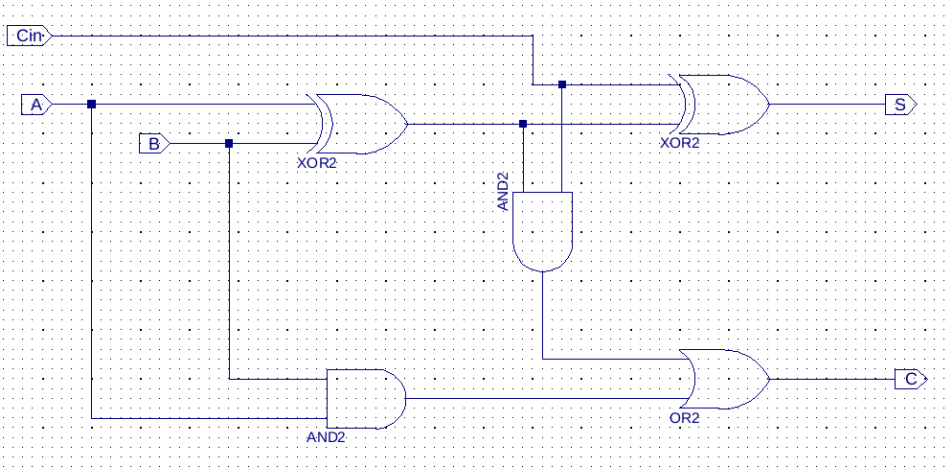
تصویر ۱۰ – مراحل ساخت **FullAdder**



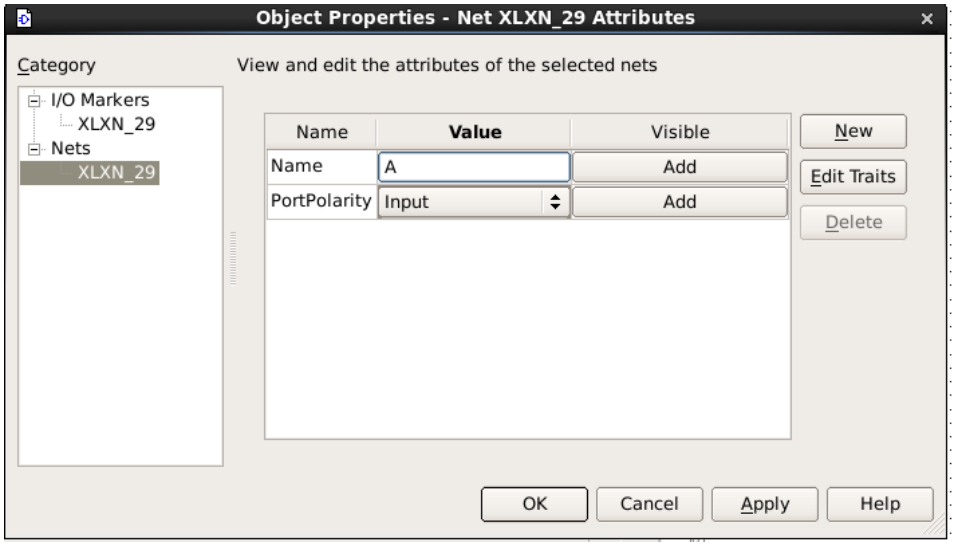
تصویر ۱۱ – مراحل ساخت **FullAdder**



تصویر ۱۲ – ایجاد پورت



تصویر ۱۳ – تکمیل طراحی یک مالتی پلکسر ۱×٤

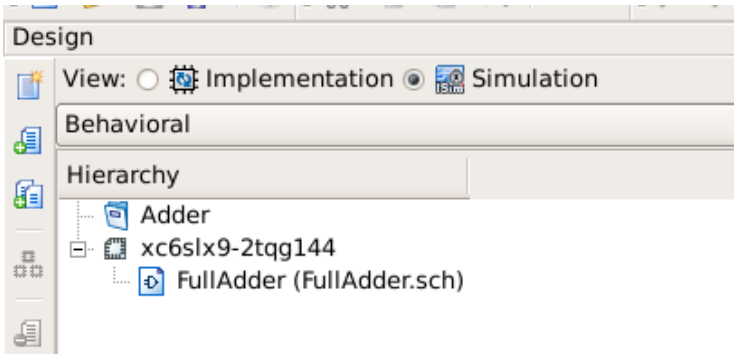


تصویر ١٤ – تنظیمات پورت

بخش ۲- انجام شبیه‌سازی رفتاری

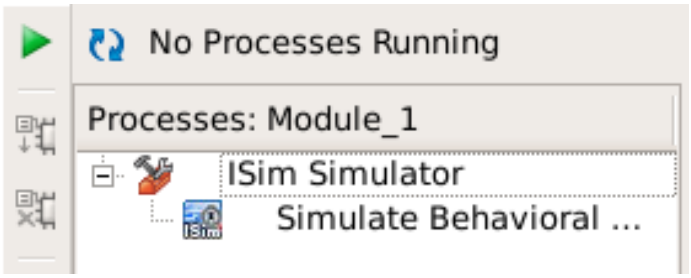
در این مرحله، به منظور تایید نتایج خروجی، مدار طراحی شده در سطح توصیف رفتاری شبیه‌سازی[۳۰] می‌شود. بدین منظور مراحل زیر باید طی شود:

۱. با توجه به تصویر ۱۵ ، در بخش View، قسمت Simulation را انتخاب کنید



تصویر ۱۵- بخش View

۲. سپس با توجه به تصویر ۱۶، در بخش Process Window، در قسمت Isim Simulator بر روی Simulate Behavioral کلیک کنید تا نرم‌افزار ISIM باز شود (در صورتی که موارد گفته شده مشاهده نشد، در Source Window بر روی فایل شماتیک خود کلیک کنید و مجددا این بخش را انجام دهید).



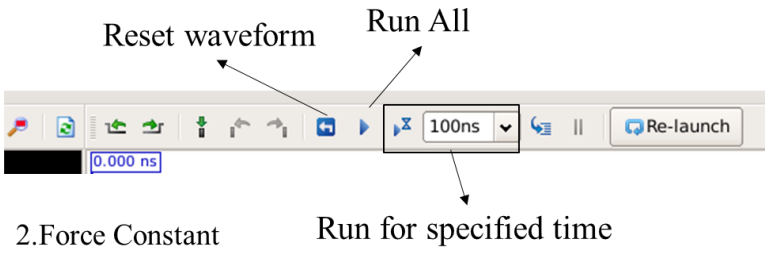
تصویر ۱۶- مراحل شبیه‌سازی

۳. در نرم‌افزار ISIM چند بخش مختلف وجود دارد که با توجه به تصویر ۱۷ و ۱۸، هر کدام را به صورت جداگانه توضیح می‌دهیم:

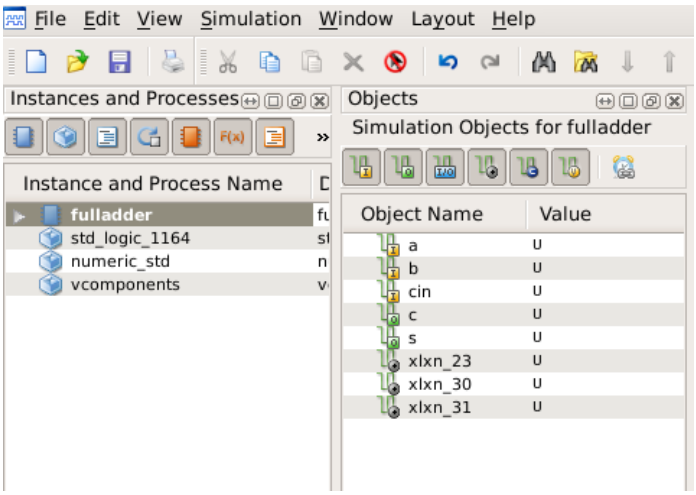
تصویر ۱۷: ابزار کنترل زمان در شبیه‌سازی

[30] Behavioral Mode Simulation

تصویر ۱۸: شامل بخش‌های Instance and Process Name و Simulation Objects for ... است در آن ساختار درختی ماژول‌های شبیه‌سازی شده و سیگنال‌های موجود در آن قابل نمایش هستند.



تصویر ۱۷- پنجره‌های مهم در ISIM

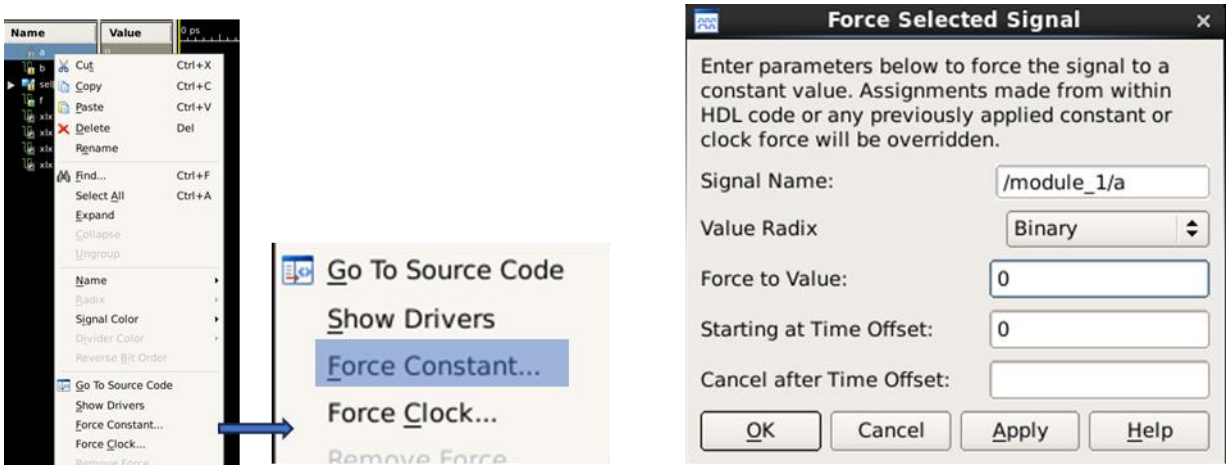


تصویر ۱۸- پنجره‌های مهم در ISIM

۴. در تصویر ۱۹ شکل موج‌های پورت‌های ورودی و خروجی نشان داده شده است. به منظور مقداردهی به یک سیگنال یا پورت، مطابق تصویر ۲۰، ابتدا بر روی سیگنال مورد نظر کلیک راست کنید و سپس در بخش Force Constant، در قسمت Force to value، مقدار اولیه‌ی خود را وارد کنید.



تصویر ۱۹- شبیه‌سازی در ISIM



تصویر ۲۰- شبیه‌سازی در ISIM

۵. بعد از وارد نمودن مقادیر دلخواه خود، در بخش تنظیم زمان (تصویر ۱۷)، زمان مورد نظر را بر روی 10 ns قرار داده و بر روی Run for specified کلیک کنید تا شکل موج نشان داده شود.

۶. با توجه به توضیحات، به سیگنال‌های موجود مقدار دلخواه را بدهید و شکل موج خروجی را تعیین کنید.

۷. در انتها، می‌توانید در منوی بالای نرم‌افزار، بر روی File کلیک کرده و فایل شبیه‌سازی خود را ذخیره کنید.

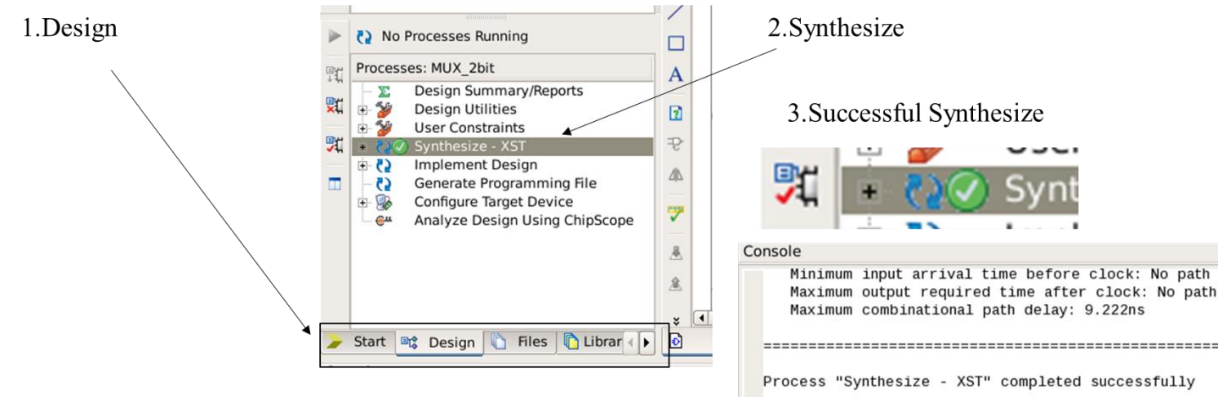
بخش ۳- انجام سنتز

گیت‌های منطقی توصیف شده در مراحل قبل، ابتدا بهینه شده یا با سایر المان‌های مداری جایگزین می‌شوند و در انتها، یک نت لیست از این المان‌ها تولید می‌شود. به منظور انجام سنتز، روال زیر باید طی شود:

۱. با توجه به تصویر ۱۵، بخش Implementation را انتخاب کنید. سپس در پنجره‌ی Source Window فایل شماتیک را انتخاب کنید.

۲. مطابق تصویر ۲۱، وارد تب Design شوید. سپس در پنجره‌ی Process Window بر روی Synthesize کلیک کنید. کمی صبر کنید تا فرآیند سنتز انجام شود.

۳. در صورت صحیح بودن سنتز، در پنجره‌ی Transcription عبارت "Process "Synthesize – XST" complete successfully" – یا در کنار عبارت Synthesize، یک تیک سبز مشاهده می‌شود.

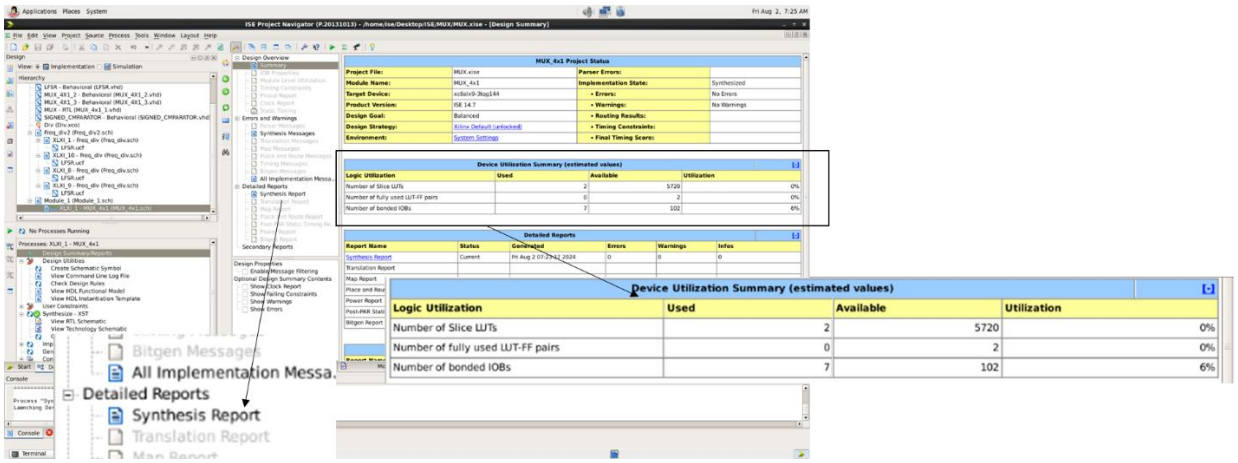


تصویر ۲۱ – انجام مرحله‌ی سنتز

۴. با توجه به تصویر ۲۱، در پنجره‌ی Process Window بر روی عبارت Design Summary/Reports کلیک کنید تا تصویر ۲۲ نمایش داده شود. با توجه به این تصویر، در دو بخش می‌توانید نتایج سنتز را مشاهده کنید:

a. جدول Design Utilization Summary: مشاهده‌ی خلاصه‌ای از تعداد منابع مصرفی

b. کلیک بر روی Synthesis Reports: مشاهده‌ی توضیحات بیشتر در ارتباط با تعداد منابع مصرفی



تصویر ۲۲ – مشاهده‌ی خلاصه‌ای از تعداد منابع مصرفی در مرحله‌ی سنتز

۵. بر روی علامت Synthesis Reports کلیک کنید تا نتایج زیر را بیابید. سپس آن‌ها را تحلیل و نتایج حاصل شده را گزارش کنید.

نتایج:

Part 1:

```
*                            Design Summary                                 *

Top Level Output File Name         : FullAdder.ngc

Primitive and Black Box Usage:
# BELS                             : 5
#      AND2                        : 2
#      OR2                         : 1
#      XOR2                        : 2
# IO Buffers                       : 5
#      IBUF                        : 3
#      OBUF                        : 2

Device utilization summary
```

Part 2:

```
Selected Device : 6slx9tqg144-2
Slice Logic Utilization:

Slice Logic Distribution:
 Number of LUT Flip Flop pairs used:      0
   Number with an unused Flip Flop:       0  out of      0
   Number with an unused LUT:             0  out of      0
   Number of fully used LUT-FF pairs:     0  out of      0
   Number of unique control sets:         0

IO Utilization:
 Number of Ios:                           5
 Number of bonded IOBs:                   5  out of    102     4%

Specific Feature Utilization:
```

Part 3:

```
Timing Report

NOTE: THESE TIMING NUMBERS ARE ONLY A SYNTHESIS ESTIMATE.
      FOR ACCURATE TIMING INFORMATION PLEASE REFER TO THE TRACE REPORT
      GENERATED AFTER PLACE-and-ROUTE.

Clock Information:
No clock signals found in this design

Asynchronous Control Signals Information:
No asynchronous control signals found in this design

Timing Summary:
```

```
Speed Grade: -2

   Minimum period: No path found
   Minimum input arrival time before clock: No path found
   Maximum output required time after clock: No path found
   Maximum combinational path delay: 9.182ns

Timing Details:
All values displayed in nanoseconds (ns)
```

Part 4:

```
Timing constraint: Default path analysis
  Total number of paths / destination ports: 8 / 2
Delay:               9.182ns (Levels of Logic = 5)
  Source:            B (PAD)
  Destination:       C (PAD)

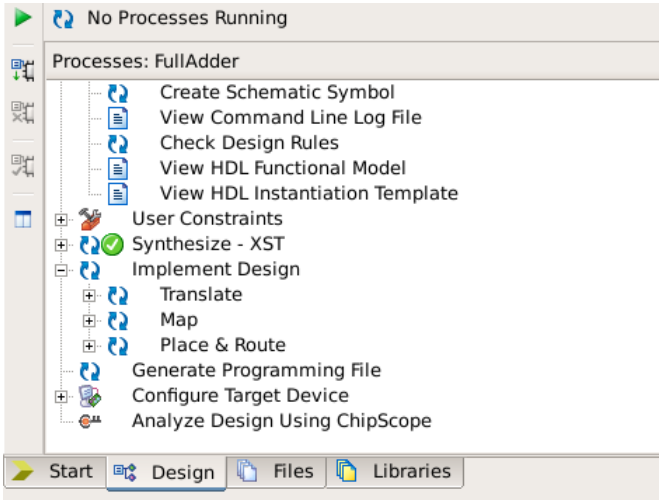
  Data Path: B to C
                                Gate     Net
    Cell:in->out      fanout   Delay   Delay  Logical Name (Net Name)
    ----------------------------------------  ------------
     IBUF:I->O             2   1.328   1.181  B_IBUF (B_IBUF)
     XOR2:I0->O            2   0.254   1.181  XLXI_1 (XLXN_23)
     AND2:I0->O            1   0.254   1.112  XLXI_6 (XLXN_30)
     OR2:I1->O             1   0.279   0.681  XLXI_5 (C_OBUF)
     OBUF:I->O                 2.912          C_OBUF ©
    ----------------------------------------
    Total                      9.182ns (5.027ns logic, 4.155ns route)
                                       (54.7% logic, 45.3% route)
```

بخش ۳- انجام پیاده‌سازی (Implementation)

این مرحله به منظور پیاده‌سازی مدار سنتز شده انجام می‌شود که شامل دو مرحله‌ی Map و Place and Route است. در اولین مرحله، مدار موجود در نت لیست، بر روی سلول‌های منطقی و ورودی/خروجی‌های تراشه نگاشت می‌شوند. سپس در مرحله‌ی بعد، یک جانمایی[۳۱] فیزیکی، بر اساس تراشه‌ی مورد نظر، استخراج می‌شود. به منظور مشاهده‌ی این عملیات در نرم‌افزار ISE، پس از انتخاب پروژه در پنجره‌ی Source Window، در پنجره‌ی Process Window گزینه‌ی Implement Design را مطابق تصویر ۲۳ بیابید و بر روی علامت مثبت کنار آن کلیک کنید. به منظور انجام پیاده‌سازی، بر روی Implement Design کلیک کنید. در صورت موفقیت‌آمیز بودن آن، در کنار این گزینه یک تیک سبز ظاهر می‌شود. به منظور مشاهده‌ی نتایج، با توجه به بخش قبل و تصویر ۲۱، در پنجره‌ی Process Window بر روی عبارت Design Summary/Reports کلیک کنید تا تصویر ۲۲ نمایش داده شود. سپس در بخش Detailed Report،

[31] Layout

گزارش‌های Place and Route Report و Post-PAR Static Timing Report را مشاهده کرده و نتایج را تحلیل کنید.



تصویر ۲۳ – انجام مرحله‌ی پیاده‌سازی

نتایج:

Part 1:

```
Device Utilization Summary:

Slice Logic Utilization:
  Number of Slice Registers:                     0 out of  11,440    0%
  Number of Slice LUTs:                          3 out of   5,720    1%
    Number used as logic:                        3 out of   5,720    1%
      Number using O6 output only:               3
      Number using O5 output only:               0
      Number using O5 and O6:                    0
      Number used as ROM:                        0
    Number used as Memory:                       0 out of   1,440    0%

Slice Logic Distribution:
  Number of occupied Slices:                     2 out of   1,430    1%
  Number of MUXCYs used:                         0 out of   2,860    0%
  Number of LUT Flip Flop pairs used:            3
    Number with an unused Flip Flop:             3 out of       3  100%
    Number with an unused LUT:                   0 out of       3    0%
    Number of fully used LUT-FF pairs:           0 out of       3    0%
    Number of slice register sites lost
      to control set restrictions:               0 out of  11,440    0%
```

Part 2:

```
IO Utilization:
  Number of bonded IOBs:                         5 out of     102    4%

Specific Feature Utilization:
  Number of RAMB16BWERs:                         0 out of      32    0%
  Number of RAMB8BWERs:                          0 out of      64    0%
  Number of BUFIO2/BUFIO2_2CLKs:                 0 out of      32    0%
```

Number of BUFIO2FB/BUFIO2FB_2CLKs:	0 out of	32	0%
Number of BUFG/BUFGMUXs:	0 out of	16	0%
Number of DCM/DCM_CLKGENs:	0 out of	4	0%
Number of ILOGIC2/ISERDES2s:	0 out of	200	0%
Number of IODELAY2/IODRP2/IODRP2_MCBs:	0 out of	200	0%
Number of OLOGIC2/OSERDES2s:	0 out of	200	0%
Number of BSCANs:	0 out of	4	0%
Number of BUFHs:	0 out of	128	0%
Number of BUFPLLs:	0 out of	8	0%
Number of BUFPLL_MCBs:	0 out of	4	0%
Number of DSP48A1s:	0 out of	16	0%
Number of ICAPs:	0 out of	1	0%
Number of MCBs:	0 out of	2	0%
Number of PCILOGICSEs:	0 out of	2	0%
Number of PLL_ADVs:	0 out of	2	0%
Number of PMVs:	0 out of	1	0%
Number of STARTUPs:	0 out of	1	0%
Number of SUSPEND_SYNCs:	0 out of	1	0%

Part 3:

Data Sheet report:

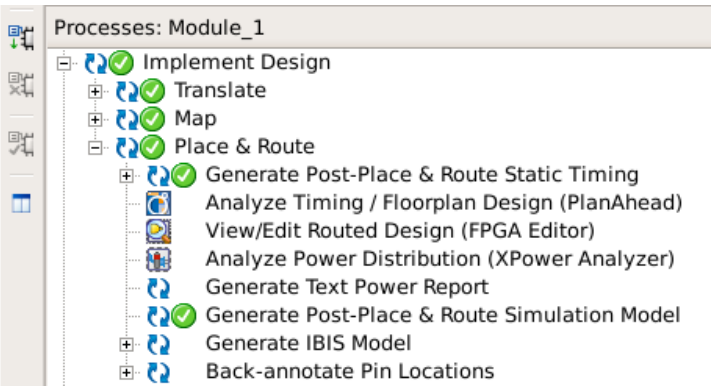
All values displayed in nanoseconds (ns)

Pad to Pad

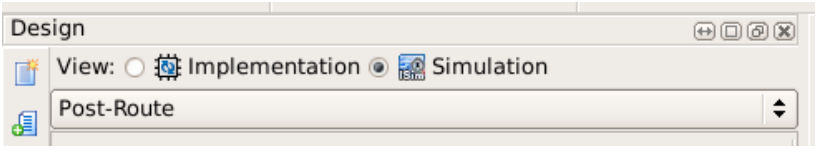
Source Pad	Destination Pad	Delay
A	C	9.553
A	S	9.316
B	C	9.611
B	S	9.374
Cin	C	8.360
Cin	S	8.310

بخش ٤- شبیه‌سازی زمانی

در این مرحله، شبیه‌سازی به منظور تایید صحت زمانی مدار انجام می‌شود. به عنوان نمونه، از صحت عملکرد مدار مورد نظر در یک فرکانس خاص اطمینان حاصل می‌شود. همچنین می‌توان سایر موارد دیگر، مانند تاخیر انتشار یا glitch را بررسی کرد. بدین منظور، پس از مرحله‌ی Implementation، در بخش Process Window، مطابق تصویر ٢٤، بر روی گزینه‌ی Generate Post-Place & Route Simulation Model کلیک کنید تا مدل شبیه‌سازی زمانی ساخته شود. سپس در بخش View، مطابق تصویر ٢٥، ابتدا گزینه‌ی Simulation و سپس Route-Post را انتخاب کنید. مطابق آن چه قبلا ذکر شده است، شبیه‌سازی را انجام دهید و نتایج را مقایسه کنید.



تصویر ۲۴ – انجام مراحل شبیه‌سازی در سطح Post-Place and Route

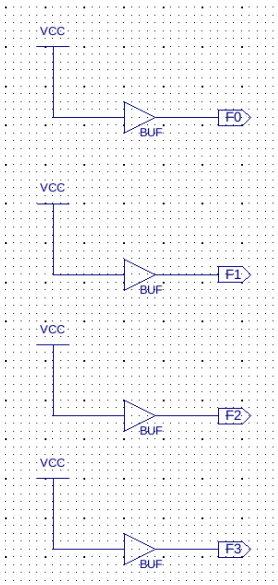


تصویر ۲۵ – انجام مراحل شبیه‌سازی سطح Post-Place and Route – بخش ۲

بخش ۵- پیکربندی بر روی برد

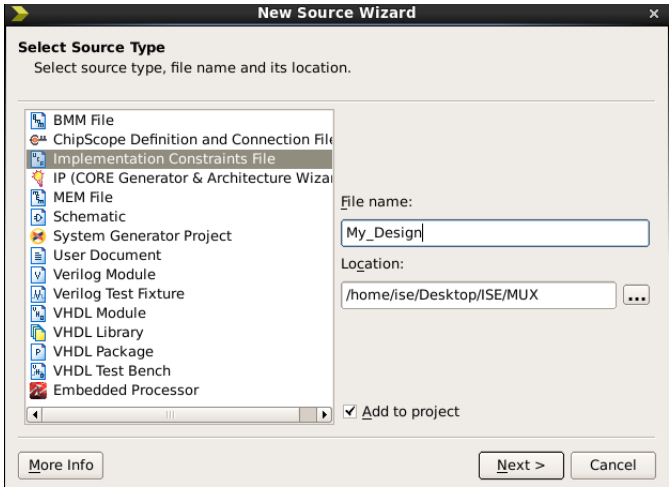
هدف از این بخش پیکربندی مدار بر روی برد پازج است. بدین منظور، مراحل زیر را طی کنید:

۱) در محیط شماتیک مدار زیر را اضافه کنید (این مدار به منظور غیرفعال کردن پایه‌های فعال‌ساز سون‌سگمنت مالتی‌پلکسری در بال مگاوینگ است). سپس مدار را سنتز کنید تا از صحت عملکرد آن اطمینان حاصل کنید.



تصویر ۲۶ – مدار غیرفعال‌سازی فعال‌سازی سون‌سگمنت مالتی‌پلکسری

۲) در این مرحله باید پورت‌های ورودی/خروجی به مدار به پایه‌های برد نگاشت شود. بدین منظور، بر روی New Source (چنان قبلا ذکر شده است) کلیک کنید. سپس با توجه به تصویر ۲۷، بر روی Implementation Constraint File کلیک کنید.



تصویر ۲۷ – انجام روال پیکربندی بر روی برد

۳) در مرحله‌ی بعد به کمک توضیحات مسئول آزمایشگاه، مدار را بر روی برد پیاده‌سازی کرده و به صورت عملی تست کنید. آدرس تمامی پین‌های برد در صفحه‌ی بعد قرار داده شده است.

گزارش کار

آزمایش انجام شده را مرور کنید و موارد خواسته شده را در گزارش‌کار خود یادداشت کنید:

۱) شاخص‌های مدار طراحی شده در این آزمایش را محاسبه کنید (تعداد المان‌ها، تاخیر مسیر بحرانی و میزان کمینه‌ی فرکانس پالس ساعت). سپس گزارش سنتز و پیاده‌سازی را بررسی کرده و تعداد المان‌ها مدار، بیشترین میزان تاخیر و فرکانس سیگنال پالس ساعت را ذکر کنید.

۲) با توجه به این آزمایش، فایل شماتیک جدیدی درست کنید و یک مدار مالتی‌پلکسر ۲ در ۱ را طراحی کنید. سپس تمامی مراحل این آزمایش را انجام دهید و نتایج آن را در گزارش‌کار خود بیاورید.

```
# UCF file for the Posedge LogiX1 megawing board - Rev 1.2

# megawing pin [] to FPGA pin Pxx map
# -------W2--------    -----W1.A-------     ------W1.B------
# [GND] [W3_8] P111    [GND] [ VS ] P85     P87 [ HS ]
# [2V5] [W3_7] P112    [2V5] [ B0 ] P83     P84 [ B1 ]
# [3V3] [W3_6] P114    [3V3] [ G0 ] P81     P82 [ G1 ]
# [5V0] [W3_5] P115    [5V0] [ R0 ] P79     P80 [ R1 ]
#       [W3_4] P116          [PB1 ] P75     P78 [PB0 ] [5V0]
#       [W3_3] P117          [PB2 ] P67     P74 [PB3 ] [3V3]
#       [W3_2] P118          [DIP7] P59     P66 [DIP6] [2V5]
#       [W3_1] P119          [DIP4] P57     P58 [DIP5] [GND]
# [GND] [LED7] P120    [GND] [DIP2] P55     P56 [DIP3]
# [2V5] [LED6] P121    [2V5] [DIP1] P48     P51 [DIP0]
# [3V3] [LED5] P123    [3V3] [an0 ] P46     P47 [seg0]
# [5V0] [LED4] P124    [5V0] [seg5] P44     P45 [an1 ]
#       [LED3] P126          [an2 ] P41     P43 [seg1] [5V0]
#       [LED2] P127          [seg4] P35     P40 [seg3] [3V3]
#       [LED1] P131          [seg7] P33     P34 [seg2] [2V5]
#       [LED0] P132          [seg6] P30     P32 [an3 ] [GND]

# Clock and Reset pins
NET "CLK"         LOC = P50;   # 24MHz on-board clock
NET "RST_N"     LOC = p133;  # active-low USER_PB button

# dip-switches
NET "DIP[0]"    LOC = P51;
NET "DIP[1]"    LOC = P48;
NET "DIP[2]"    LOC = P55;
NET "DIP[3]"    LOC = P56;
NET "DIP[4]"    LOC = P57;
NET "DIP[5]"    LOC = P58;
NET "DIP[6]"    LOC = P66;
NET "DIP[7]"    LOC = P59;

# Push-Buttons
NET "PB[0]"     LOC = P78;
NET "PB[1]"     LOC = P75;
NET "PB[2]"     LOC = P67;
NET "PB[3]"     LOC = P74;

# LEDs
NET "LED[7]"      LOC = P120;
NET "LED[6]"      LOC = P121;
NET "LED[5]"      LOC = P123;
NET "LED[4]"      LOC = P124;
NET "LED[3]"      LOC = P126;
NET "LED[2]"      LOC = P127;
NET "LED[1]"      LOC = P131;
NET "LED[0]"      LOC = P132;

# 7-segment
NET "an[0]"     LOC = P46;
NET "an[1]"     LOC = P45;
NET "an[2]"     LOC = P41;
NET "an[3]"     LOC = P32;
NET "sseg[0]"   LOC = P47;
```

```
NET "sseg[1]"   LOC = P43;
NET "sseg[2]"   LOC = P34;
NET "sseg[3]"   LOC = P40;
NET "sseg[4]"   LOC = P35;
NET "sseg[5]"   LOC = P44;
NET "sseg[6]"   LOC = P30;
NET "sseg[7]"   LOC = P33;

# VGA
NET "HS"        LOC = P87;
NET "VS"        LOC = P85;
NET "R[0]"      LOC = P79;
NET "R[1]"      LOC = P80;
NET "G[0]"      LOC = P81;
NET "G[1]"      LOC = P82;
NET "B[0]"      LOC = P83;
NET "B[1]"      LOC = P84;

# SD-CARD
NET "CS"          LOC = P111;
NET "DIN"         LOC = P112;
NET "CLK"         LOC = P114;
NET "DOUT"        LOC = P115;

# WING 3
NET "W3_4"        LOC = P116;
NET "W3_3"        LOC = P117;
NET "W3_2"        LOC = P118;
NET "W3_1"        LOC = P119;
```

پروژه–۱: طراحی جمع‌کننده‌ی ٤ بیت به کمک الگوریتم Ripple-Carry Adder و Ripple Look Ahead Adder

دو الگوریتم مهم به منظور طراحی یک جمع کننده وجود دارد: الگوریتم Ripple-Carry Adder و Carry Look Ahead Adder. در مورد هر دو تحقیق کنید و سپس به سوالات زیر پاسخ دهید:

۱. ساختار این دو الگوریتم را از لحاظ شاخص‌های سخت‌افزاری با یکدیگر مقایسه کنید.
۲. به کمک الگوریتم اول، یک جمع‌کننده‌ی ٤ بیت طراحی کنید. به منظور صحت عملکرد، آن را شبیه‌سازی کنید. سپس تمامی مراحل گفته شده در آزمایش را انجام دهید و نتایج آن را در یک جدول ذکر کنید.
۳. الگوریتم Carry Look Ahead Adder به کمک سمبل CARRY4 در ابزار شماتیک قابل پیاده‌سازی است. بدین منظور، با مراجعه به دیتاشیت تراشه‌ی Spartan 6، روش استفاده از آن را بیابید.
۴. تمامی نتایج به دست آمده از مراحل دو مرحله‌ی ۲ و ۳ را با یکدیگر مقایسه کنید.
۵. مدار توصیف شده در بخش ۳ را بر روی برد پیاده‌سازی کنید.

آزمایش شماره‌ی ۲

هدف:

- آشنایی با زبان برنامه‌نویسی VHDL و توصیف مدارهای ترکیبی در سطح گیت

تئوری آزمایش:

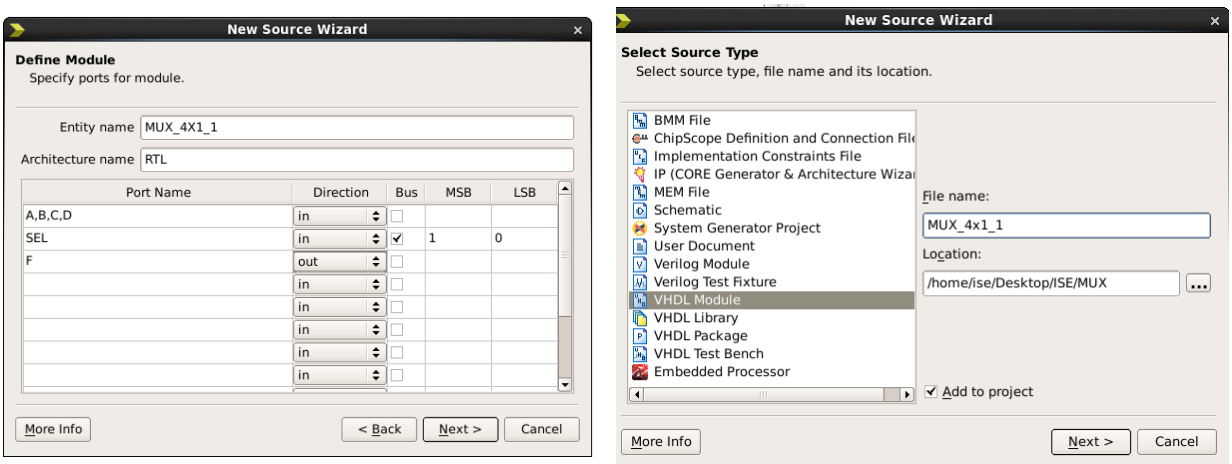
در این آزمایش، به ترتیب سه مرحله را طی خواهیم کرد. در مرحله‌ی اول، یک کد VHDL ایجاد می‌کنیم و سپس با ساختار کد VHDL آشنا می‌شویم. در ادامه، در محیط کدنویسی، مدار یک مالتی پلکسر ۱×۴ را کد نویسی می‌کنیم. در مرحله‌ی دوم، مراحل سنتز تا پیاده‌سازی را انجام خواهیم داد. در مرحله‌ی آخر، مدار را به کمک نرم‌افزار ISIM شبیه سازی کرده و پس از صحت عملکرد مدار، آن را بر روی برد پیکربندی می‌کنیم.

تمام کدهای این آزمایش را در پروژه‌ای به نام EX2 ذخیره کنید.

آزمایش:

قدم اول – ایجاد کد VHDL

۱. با توجه به آزمایش قبل، به بخش New Source بروید و مشابه تصویر ۱–الف، یک ماژول VHDL بسازید و پورت‌های ورودی و خروجی مدار را مشابه تصویر ۱–ب قرار دهید.



ب الف

تصویر ۱ – ساخت ماژول VHDL

۲. کد تولید شده را مشابه حالت زیر تغییر بدهید:

```
1   library IEEE;
2   use IEEE.STD_LOGIC_1164.ALL;
3
4   entity MUX_4X1_1 is
5       Port ( A,B,C,D : in  STD_LOGIC;
6              SEL : in  STD_LOGIC_VECTOR (1 downto 0);
```

```
7               F : out  STD_LOGIC);
8   end MUX;
9
10  architecture RTL of MUX_4X1_1 is
11  SIGNAL M1,M2,M3,M4 : STD_LOGIC;
12  begin
13  M1 <= (not SEL(0)) AND (not SEL(1)) AND A;
14  M2 <= SEL(0) AND (not SEL(1)) AND B;
15  M3 <= (not SEL(0)) AND SEL(1) AND C;
16  M4 <= SEL(0) AND SEL(1) AND D;
17  F <= M1 OR M2 OR M3 OR M4;
18
19  end RTL;
```

۳. کد را شرح بدهید و مدار منطقی آن را بر روی کاغذ ترسیم کنید.

قدم دوم – مراحل سنتز و پیاده‌سازی

۱. کد VHDL مرحله‌ی قبل را سنتز کنید. سپس نتایج حاصل شده را گزارش دهید.

۲. پس از سنتز، مراحل پیاده‌سازی را انجام داده و گزارش‌های حاصل شده را گزارش کنید.

قدم سوم و چهارم – شبیه‌سازی و پیکربندی

۱. مشابه آزمایش قبل، کد را در دو سطح رفتاری و Post-Place and Route شبیه‌سازی کرده و نتایج را گزارش کنید.

۲. کد را بر روی برد پیاده‌سازی کنید.

گزارش‌کار

تمامی گزارش‌های ذکر شده در کل آزمایش را در گزارش‌کار خود آورده و نتایج آن را تحلیل کنید.

بخش سوم – طراحی مدارهای منطقی به کمک برنامه‌نویسی VHDL

هدف این بخش استفاده از منابع سخت‌افزاری برد پازج و پیاده‌سازی مدارهای منطقی به کمک آن‌هاست. لذا در آزمایش‌های مختلف، به کمک ساختارهای مختلف کدنویسی VHDL، مدارهای مختلفی را کدنویسی، شبیه‌سازی و سپس بر روی برد پیکربندی می‌کنیم.

آزمایش شماره‌ی ۳

هدف:

- آشنایی با دستور When/Select
- آشنایی با ساختار Seven Segmentهای برد آزمایشگاه
- طراحی و پیاده‌سازی مدار تبدیل کد BCD به Seven Segment

تئوری آزمایش:

بر روی بردهای FPGA موجود در آزمایشگاه، چهار سون سگمنت وجود دارد که از نوع آند مشترک هستند. در این آزمایش می‌خواهیم مدارهایی را طراحی کنیم که ابتدا کد BCD را از ورودی گرفته و بر روی سون سگمنتی که انتخاب می‌کنیم، نمایش بدهد. بدین منظور، در این آزمایش سه مرحله طی خواهد شد. در مرحله‌ی اول، ابتدا جدول درستی مدار تبدیل BCD به سون سگمنت ترسیم می‌شود. سپس به کمک این جدول، کد VHDL این مدار نوشته می‌شود. در مرحله‌ی دوم، مشابه مرحله‌ی اول، برای پایه‌ی انتخابگر سون سگمنت‌ها یک کد VHDL نوشته می‌شود. در آخرین مرحله، به کمک مراحل قبل، مداری توصیف می‌شود که کد BCD را به از ورودی دریافت کرده و با کمک پایه‌ی انتخابگر، بر روی سون سگمنت مورد نظر نمایش دهد.

تمام کدهای این آزمایش را در پروژه‌ای به نام EX3 ذخیره کنید.

آزمایش

قدم ۱ – طراحی مدار تبدیل کد BCD به سون سگمنت

۱) با توجه به سون سگمنت آند مشترک، جدول زیر را کامل کنید.

ورودی (کد BCD)				خروجی							
$X(3)$	$X(2)$	$X(1)$	$X(0)$	h	g	f	e	d	c	b	a
0	0	0	0	1	1	0	0	0	0	0	0
0	0	0	1								

Others				1	1	1	1	1	1	1	1

۲) با توجه به جدول بالا و کدهای توصیف شده در بخش پیش‌گزارش، مدار تبدیل کد BCD به سون سگمنت را طراحی کنید. در این حالت فرض کنید که خروجی مدار یک سیگنال ۸ بیت است که بیت ۰ تا ۷ به ترتیب پورت‌های a تا h هستند.

۳) مدار را در سطح رفتاری شبیه‌سازی کنید.

قدم ۲ – طراحی مدار انتخابگر سون سگمنت

سون سگمنت‌هایی که بر روی بردهای FPGA موجود در آزمایشگاه قرار دارند، دارای پایه‌ی فعال‌ساز مخصوص به خود هستند و در صورتی که بخواهیم از یکی آن‌ها استفاده کنیم، ابتدا باید به کمک این پایه، آن را فعال کنیم. در این حالت، نیازمند مداری هستیم که دارای ٤ بیت ورودی و ٤ بیت خروجی باشد. جدول درستی این مدار به صورت زیر است. به کمک کد VHDL این مدار را ایجاد کنید.

ورودی				خروجی			
$X(3)$	$X(2)$	$X(1)$	$X(0)$	$S(3)$	$S(2)$	$S(1)$	$S(0)$
0	0	0	1	1	1	1	0
0	0	1	0	1	1	0	1
0	1	0	0	1	0	1	1
1	0	0	0	0	1	1	1
Others				Others => ‘z’			

قدم ۳ – طراحی مدار تبدیل کد BCD به سون سگمنت به همراه انتخابگر سون سگمنت

مدارهای توصیف شده در قسمت‌های ۲ و ۳ را در کنار یکدیگر قرار بدهید و مداری طراحی کنید که یک عدد BCD را از ورودی گرفته و بر روی سون سگمنت دلخواه نمایش دهد. در ادامه، مدار را بر روی برد پیاده‌سازی کرده و تست کنید.

گزارش‌کار

نتایج سنتز و Implementation را برای تمامی مدارهای موجود بررسی و تحلیل کنید. سپس شبیه‌سازی را در دو سطح Behavioral و Post and Route انجام داده و نتایج را در گزارش‌کار خود ذکر کنید.

آزمایش شماره‌ی ٤

هدف:

- طراحی مدارهای ترتیبی ثانیه‌شمار

تئوری آزمایش:

در این آزمایش سه قدم را طی خواهیم کرد. ابتدا کد VHDL یک شمارنده‌ی ٤ بیت را بررسی و شبیه‌سازی می‌کنیم. در قدم دوم با توسعه‌ی شمارنده‌ی قدم اول، یک شمارنده طراحی می‌کنیم که با فرض فرکانس 24 MHz، یک عدد به سمت بالا بشمارد. سپس در قدم آخر، مدار نهایی بر روی برد پیاده‌سازی می‌شود.

تمام کدهای این آزمایش را در پروژه‌ای به نام EX4 ذخیره کنید.

آزمایش:

قدم ۱ – طراحی شمارنده‌ی ٤ بیت

۱. کد زیر را تحلیل کنید.

```
1     library IEEE;
2     use IEEE.STD_LOGIC_1164.ALL;
3     use IEEE.NUMERIC_STD.ALL;
4     entity Counter is
5     Port ( CLK,RST : in  STD_LOGIC;
6            CNT : out  STD_LOGIC_VECTOR (3 downto 0));
7     end Counter;
8
9     architecture Behavioral of Counter is
10    SIGNAL CNT1 : UNSIGNED(3 DOWNTO 0);
11    begin
12    PROCESS(CLK)
13          BEGIN
14                IF CLK'EVENT AND CLK = '1' THEN
15                      IF RST = '1' then
16                            CNT1 <= (OTHERS => '0');
17                      ELSE
18                            CNT1 <= CNT1 + 1;
19                      END IF;
20                END IF;
21    END PROCESS;
22    CNT <= STD_LOGIC_VECTOR(CNT1);
23
24    end Behavioral;
```

۲. با فرض فرکانس کاری 24 MHz در مدار توصیف شده، کد را طوری تغییر دهید که شمارش هر یک ثانیه انجام شود.

۳. مدار را بر روی برد پیاده‌سازی کنید.

گزارش‌کار

تمامی گزارش‌های نرم‌افزار را به همراه تحلیل کدها در گزارش‌کار خود ذکر کنید (نتایج شبیه‌سازی ISIM نیازی نیست).

پروژه-۲

۱. شمارنده‌ی موجود در آزمایش ٤ را به گونه‌ای تغییر دهید که به جای ۰ تا ۱۵، از ۰ تا ۹۹۹۹ شمرده و آن را بر روی سون سگمنت‌های روی برد نشان دهد.

۲. یک مدار قدر مطلق ۸ بیت بسازید.

آزمایش شماره‌ی ۵

هدف:

- استفاده از IP-Core
- آشنایی با سیستم عددی ممیز ثابت (Fixed-Point)
- آشنایی با الگوریتم کوردیک به منظور پیاده‌سازی دیجیتال توابع مثلثاتی سینوس و کسینوس

تئوری آزمایش:

هدف از این آزمایش آشنایی با الگوریتم کوردیک و پیاده‌سازی آن بر روی تراشه‌ی FPGA با کمک ابزار آماده‌ی IP-Core است. الگوریتم کوردیک (CORDIC)، مخفف Coordinate Rotation Digital Computer، یک روش ساده و کارآمد برای انجام عملیات مثل چرخش، تبدیل قطبی-مختلط و محاسبه توابع مثل سینوس و کسینوس با استفاده از عملیات جمع و شیفت است. یکی از ابزارهای پیشرفته‌ای که برای پیاده‌سازی این الگوریتم استفاده می‌شود، استفاده از واحدهای آماده‌ی IP-Core است. بدین منظور این آزمایش در سه قدم طی می‌شود:

۱. به کمک نرم‌افزار ISE یک IP-Core الگوریتم کوردیک ساخته می‌شود.
۲. در ادامه یک کد VHDL ساخته شده و IP-Core فراخوانده می‌شود.
۳. بعد از انجام کدنویسی، مدار بر روی برد پیکربندی می‌شود و با توجه به توضیحات آزمایشگاه، برای چند زاویه‌ی خاص سینوس یا کسینوس محاسبه می‌شود.

تمام کدهای این آزمایش را در پروژه‌ای به نام EXtrigonometry ذخیره کنید.

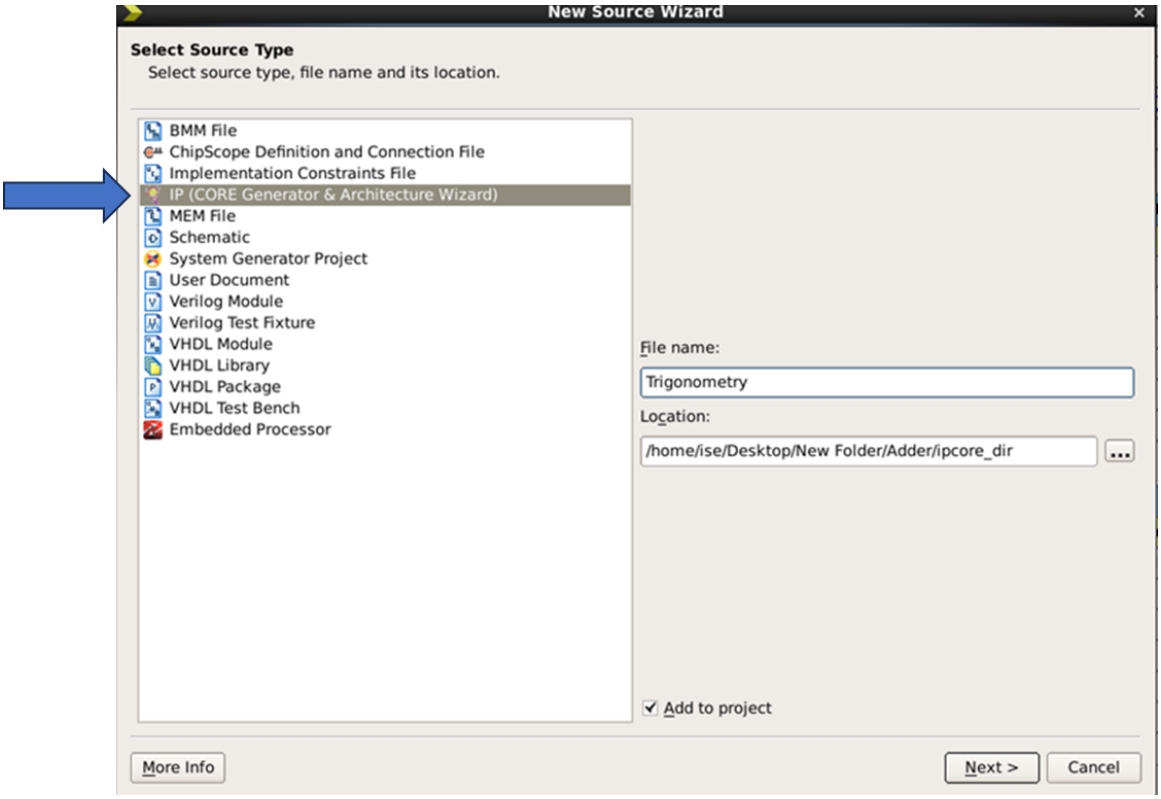
آزمایش:

قدم ۱ – ساخت IP-Core

IP-Core واحد قابل استفاده مجدد و پیکربندی‌پذیر است که عملکردهای دیجیتال معمول هستند که توسط شرکت‌های سازنده‌ی FPGA ارائه می‌شوند. مزیت این واحدها سرعت قابلیت توسعه‌ی بالا، کاهش ریسک طراحی، بهینه‌سازی‌های مربوط به مصرف توان و منابع و امکان استفاده مجدد از هسته‌ها در پروژه‌های مختلف است.

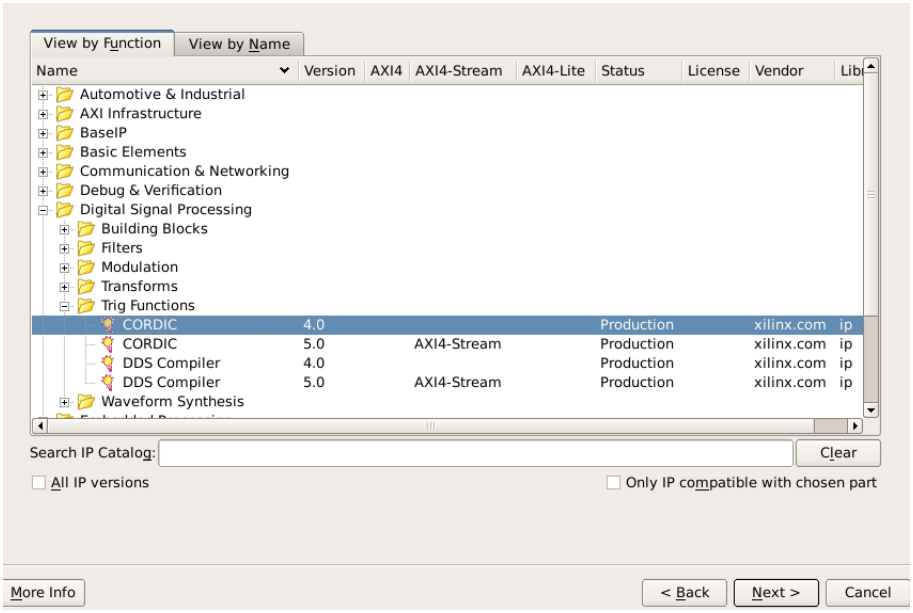
به منظور فراخوانی IP-Core مربوط به الگوریتم کوردیک، مراحل زیر را انجام دهید:

۱. پس از ساخت پروژه، ، یک Source جدید بسازید و در مرحله‌ی انتخاب نوع آن، مطابق تصویر ۱، نام را trigonometry گذاشته و در بخش نوع ماژول، بر روی IP کلیک کنید.



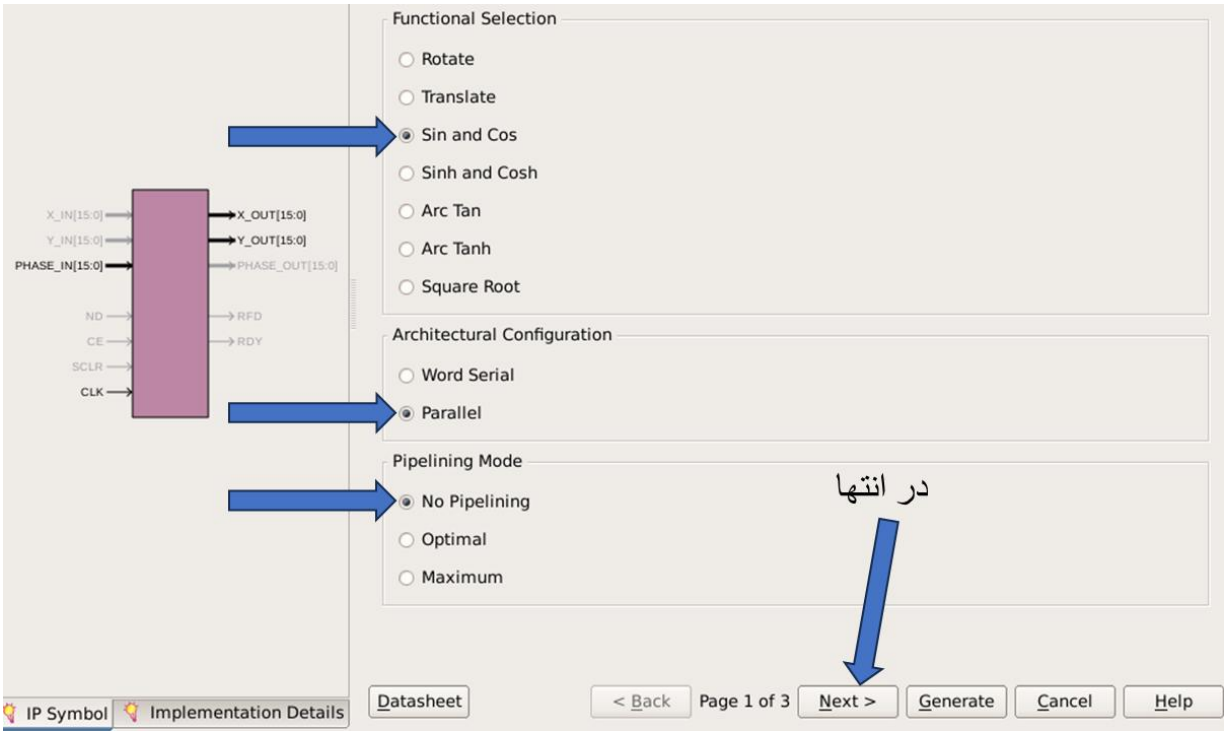
تصویر ۱ – ساخت IP-Core

۲. بعد از کلیک بر روی Next، پنجره‌ی تصویر ۲ باز می‌شود. در این پنجره، مطابق تصویر ۲، به بخش Digital Signal Processing بروید. سپس در زیربخش Trig Function، بر روی CORDIC کلیک کنید. در پنجره‌ی بعد بر روی Finish کلیک کنید.

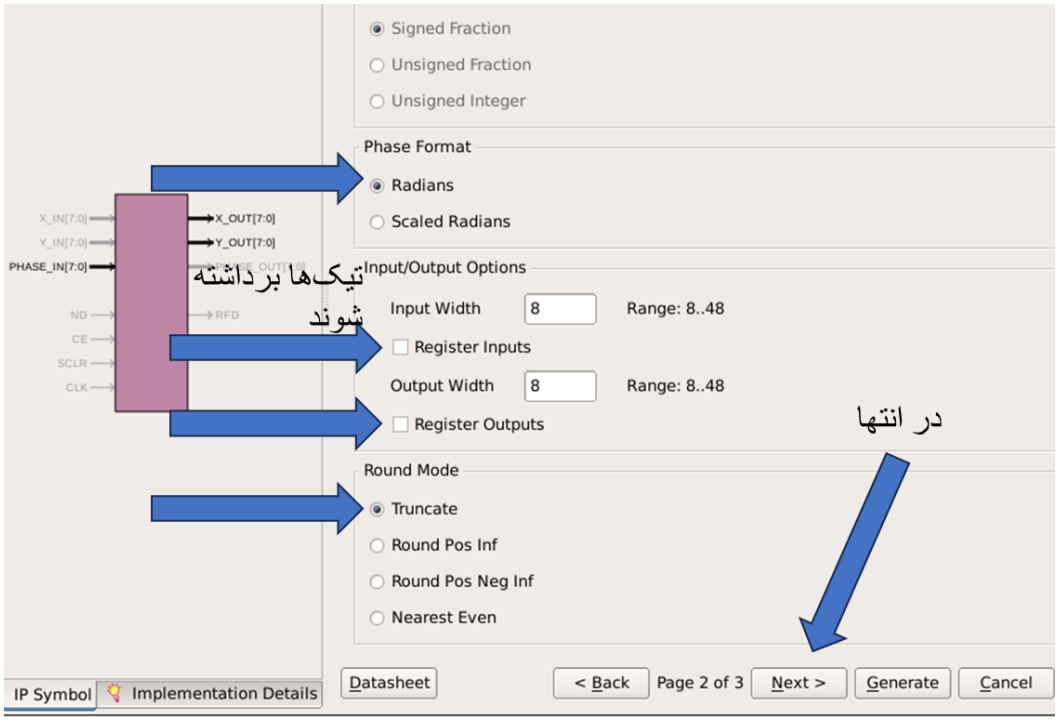


تصویر ۲ – ساخت IP-Core الگوریتم CORDIC

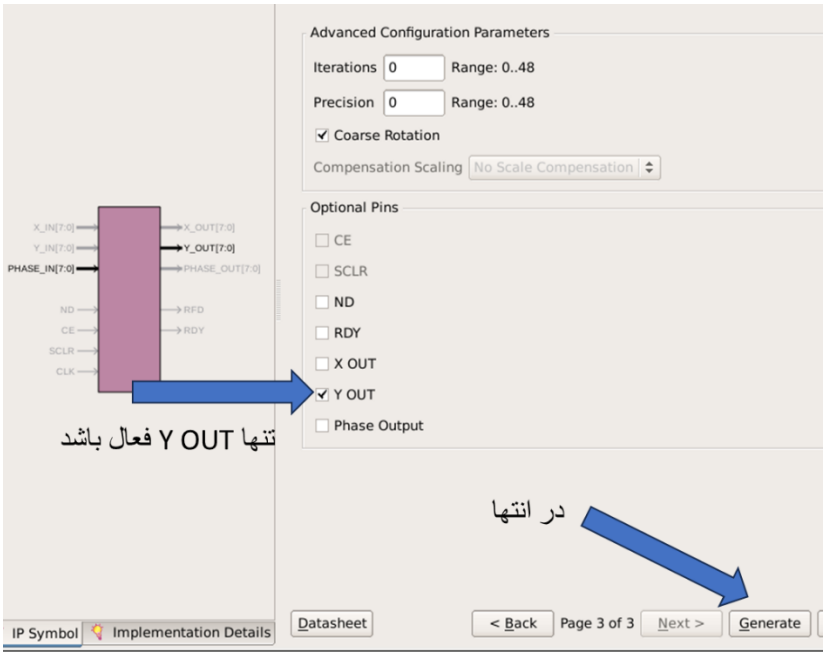
۳. بعد از کمی مکث، پنجره‌ای مطابق تصویر ۳ به منظور تنظیم IP باز می‌شود. در این پنجره، تنظیمات را مطابق تصویر ۳ تا ۵ انجام بدهید.



تصویر ۳ – ساخت IP-Core الگوریتم CORDIC



تصویر ٤ – ساخت IP-Core الگوریتم CORDIC



تصویر ۵ – ساخت IP-Core الگوریتم CORDIC

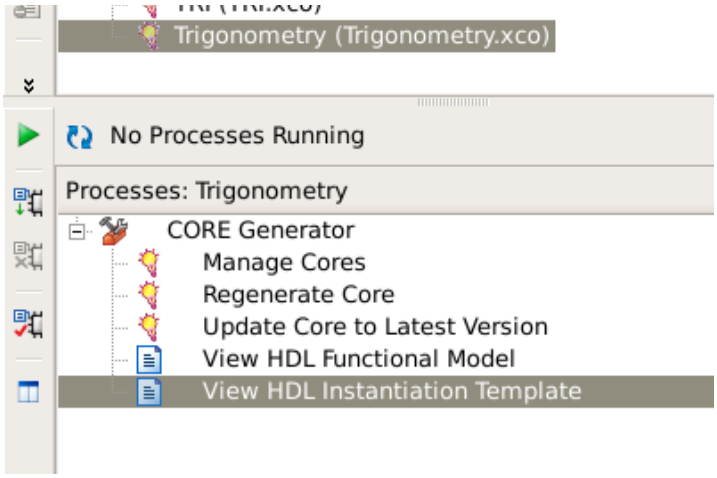
۴. پس از کمی مکث، IP-Core ساخته شده در پنجره‌ی Source Window ظاهر می‌شود. مطابق تصویر ٦، آن را انتخاب کرده و در پنجره‌ی Process Window بر روی View VHDL Instantiation Template کلیک کنید. بعد از کمی مکث، پنجره‌ای باز می‌شود که در آن عبارات زیر نوشته شده است:

Part1:

```
COMPONENT Trigonometry
  PORT (
    phase_in : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
    y_out : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
  );
END COMPONENT;
```

Part2:

```
your_instance_name : Trigonometry
  PORT MAP (
    phase_in => phase_in,
    y_out => y_out
  );
```



تصویر ۵ – ساخت IP-Core الگوریتم CORDIC

قدم ۲ و ۳– فراخوانی IP-Core

در اولین گام یک ماژول VHDL بسازید و نام فایل را Sines بنامید. سپس ورودی و خروجی را به ترتیب Phase و Sin نامیده و باس ورودی هر دو را ۸ بیت معرفی کنید (۷ تا ۰). بعد از ساخته شدن کد، کد نهایی را به صورت زیر تغییر دهید:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
library UNISIM;
use UNISIM.vComponent.ALL;


entity Sines is
    Port ( Phase : in  STD_LOGIC_VECTOR (7 downto 0);
           Sin : out  STD_LOGIC_VECTOR (7 downto 0));
end Sines;

architecture Behavioral of Sines is
COMPONENT Trigonometry
  PORT (
    phase_in : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
    y_out : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
  );
END COMPONENT;
begin
Unit : Trigonometry
  PORT MAP (
    phase_in => Phase,
```

```
    y_out => Sin
  );
end Behavioral;
```

بعد از انجام مراحل گفته شده، با توجه به توضیحات آزمایشگاه، مدار را بر روی برد پیکربندی کنید. سپس برای چند زاویه‌ی مختلف، مقدار سینوس را حساب کنید و با نتایج واقعی مقایسه کنید.

گزارش‌کار

در مورد الگوریتم کوردیک تحقیق کنید (نیازی به گزارش‌کار نیست).

مراجع مطالعاتی

مراجع لاتین:

1. Pedroni, V. A. (2020). Circuit Design with VHDl. London, England, Cambridge, MA : The MIT Press.
2. Harris, S. L. and D. M. Harris (2016). Digital Design and Computer Architecture, Morgan Kaufmann is an imprint of Elsevier.
3. Xilinx, Inc. "Spartan-6 Family Overview." 2011. https://www.xilinx.com/products/silicon-devices/fpga/spartan-6.html#documentation.
4. Xilinx, Inc. " Spartan-6 FPGA Configurable Logic Block." 2010. https://www.xilinx.com/products/silicon-devices/fpga/spartan-6.html#documentation
5. Xilinx, Inc. " Spartan-6 FPGA DSP48A1 Slice." 2014. https://www.xilinx.com/products/silicon-devices/fpga/spartan-6.html#documentation
6. Xilinx, Inc. " Spartan-6 FPGA Block RAM Resources." 2011. https://www.xilinx.com/products/silicon-devices/fpga/spartan-6.html#documentation
7. Floyd, T. L. (2015). Digital Fundamentals. England, Pearson Education Limited.
8. Yang, H., et al. (2014). "Review of advanced FPGA architectures and technologies." Journal of Electronics (China) 31(5): 371-393.
9. Pedroni, V. A. (2004). Circuit Design with VHDl. London, England, Cambridge, MA : The MIT Press.
10. Chu, P. P. (2008). FPGA PROTOTYPING BY VHDL EXAMPLES. Hoboken, New Jersey, John Wiley & Sons, Inc.
11. Chu, P. P. (2017). FPGA PROTOTYPING BY VHDL EXAMPLES. Hoboken, New Jersey, John Wiley & Sons, Inc.
12. Kaeslin, H. ( ). Top-down digital VLSI design: from architectures to gate-level circuits and FPGAS. Waltham, MA, Morgan Kaufmann is an imprint of Elsevier.
13. Meyer-Baese, U. (2007). Digital Signal Processing with Field Programmable Gate Arrays. Florida State University, College of Engineering.

مراجع فارسی:

۱۴. عجمی بهبهانی، سید محمد. (۱٤۰۰). بهینه‌سازی عملگر کانولوشن بر پایه‌ی ساختار چند پنجره‌ای. پایان‌نامه‌ی کارشناسی ارشد. جندی شاپور دزفول

۱۵. تابنده، محمود و مکی، سید محمد. (۱۳۹۵). مدارهای منطقی و سخت‌افزار کامپیوتر. تهران، موسسه‌ی انتشارات علمی دانشگاه صنعتی شریف

۱۶. برد FPGA آموزشی پازج - پازج